DineDash Food Delivery Data Anaysis

-Sathvika Kasam

1. Background

DineDash food delivery industry has seen rapid growth over the last decade, fueled by the increasing demand for convenience, improved delivery logistics, and the expansion of digital platforms. MealDash have revolutionized how food is ordered and delivered, creating new opportunities for restaurant owners and consumers alike. However, as the industry continues to expand, food delivery platforms face increasing competition, customer expectations for faster and more personalized service, and challenges in optimizing their operations across a diverse and extensive menu offering.

In this case study, we focus on the use of **big data and advanced analytics** to streamline operations, optimize delivery times, enhance customer experience, and improve menu offerings. By leveraging data processing platforms such as **Apache Spark** and **Databricks**, we can better understand customer preferences, predict demand, and offer personalized recommendations, ultimately driving more efficient and profitable business operations.

2. Challenges

Dinedash has to address several challenges to maintain competitive advantage and operational efficiency:

- **Data Fragmentation:** Different systems for orders, customer profiles, menus, and delivery logs lead to fragmented data that is difficult to analyze comprehensively.

- **Customer Segmentation:** Understanding customer preferences based on historical data to offer personalized promotions, menu recommendations, or loyalty rewards.
- **Order Prediction:** Predicting order volume in different regions at different times of the day to optimize delivery capacity and reduce wait times.
- **Menu Optimization:** Identifying the most popular and profitable menu items while suggesting item combinations to customers. Ensuring the menu is always updated in real-time for availability.
- **Order Fulfillment Delays:** Ensuring on-time deliveries and improving operational workflows, which are often impacted by external factors like weather, traffic, or restaurant delays.
- **Promotions and Discounts:** Identifying the most effective promotional offers and discounts that increase order value and drive repeat purchases without eroding profitability.

3. Business Needs

To stay competitive and meet customer demands, DineDash needs to address the following:

- **Data Integration:** Integrating data from different sources (customer orders, restaurant data, delivery times, etc.) to build a comprehensive view of customer behavior, restaurant performance, and delivery operations.
- **Real-Time Analytics:** Implementing a real-time data processing pipeline that analyzes orders as they come in, providing instant insights and enabling dynamic decision-making (e.g., dynamic pricing, offer recommendations).

- **Predictive Analytics:** Using machine learning to predict demand, customer preferences, delivery time, and possible bottlenecks in the system.
- **Optimization Algorithms:** Implementing algorithms that optimize the delivery process by calculating the best route, available delivery agents, and reducing food delivery time.
- **Personalization:** Offering personalized promotions, product recommendations, and even tailored menus based on past customer behavior, location, and preferences.
- **Performance Tracking:** Continuously monitoring key performance indicators (KPIs), such as customer satisfaction, order fulfillment rate, delivery time, and revenue per order.

4. Proposed Solution:

The proposed solution involves setting up a **big data processing pipeline** using **Databricks** and **Apache Spark**, along with **machine learning models** to provide real-time insights and predictive analytics. The solution leverages:

- **Data Lake:** A centralized repository for storing all structured and unstructured data, including order details, customer information, menu data and delivery logs.
- **ETL Pipelines**: Using **Delta Live Tables** (DLT) to manage the transformation of raw data into usable formats for analysis. The pipeline will clean, enrich, and structure the data for downstream analytics.
- **Real-Time Streaming**: Processing incoming orders in real-time using **structured streaming** to calculate metrics like order volume, delivery times, and detect any operational anomalies as they occur.
- **Analytics Dashboards**: Developing dashboards to visualize customer trends, restaurant performance, order patterns, and delivery efficiency.

- **Machine Learning Models**: Implementing models for demand forecasting, customer segmentation, and personalized recommendations to boost sales and improve customer experience.
- **Optimized Delivery Routing**: Using geographic data and algorithms to optimize delivery routes and reduce delivery times.

5. Attributes

DineDash will track the following key attributes across different tables:

1. Customers

- customer_id: Unique identifier for each customer.
- name: Name of the customer.
- email: Email address of the customer.
- signup_date: Date customer signed up

2. Restaurants

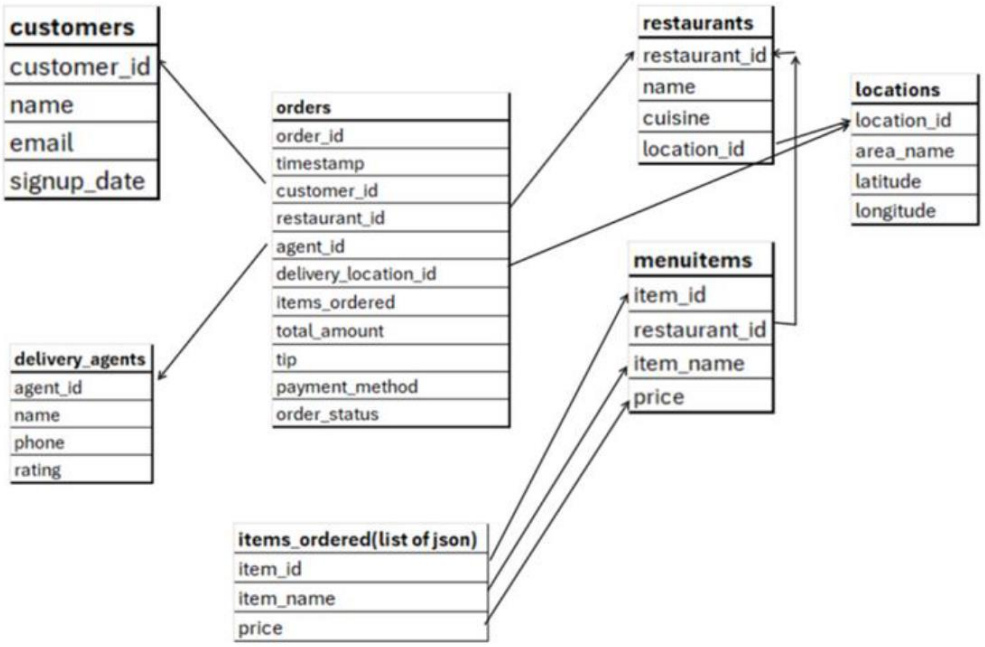
- restaurant_id: Unique identifier for each restaurant.
- name: Name of the restaurant.
- cuisine: Type of cuisine (e.g., Italian, Chinese, Indian).
- location_id: Geographic location (latitude/longitude) of the restaurant
- items_ordered: List of items in the order (can be a list of menu_item_ids).
- Total_amount:
- Tip:
- payment_method: Payment method used (e.g., "Credit Card", "Cash").
- order_status: Status of the order (e.g., "Pending", "Delivered").

5. Delivery Agent

- agent_id: Unique identifier for each delivery.

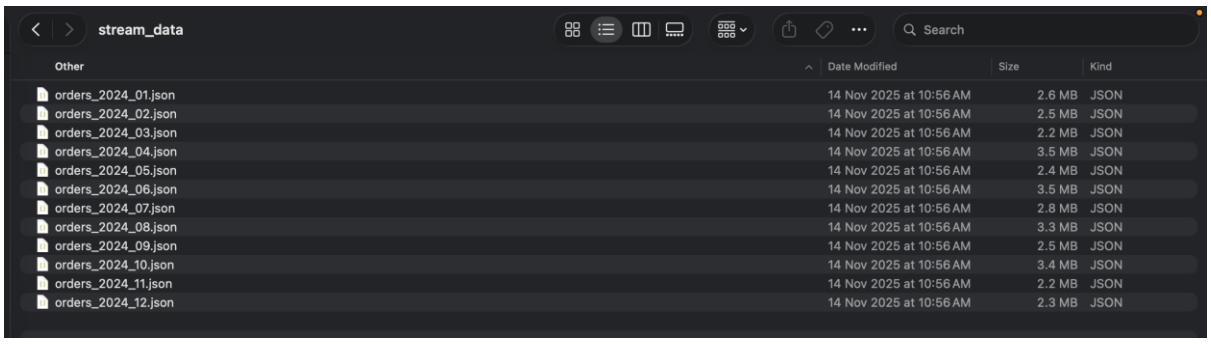
- name:
- phone:
- rating:

Data model

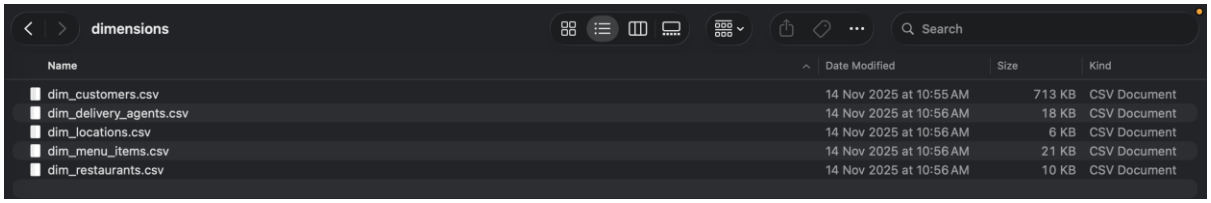


DATASETS:

Order datasets files:

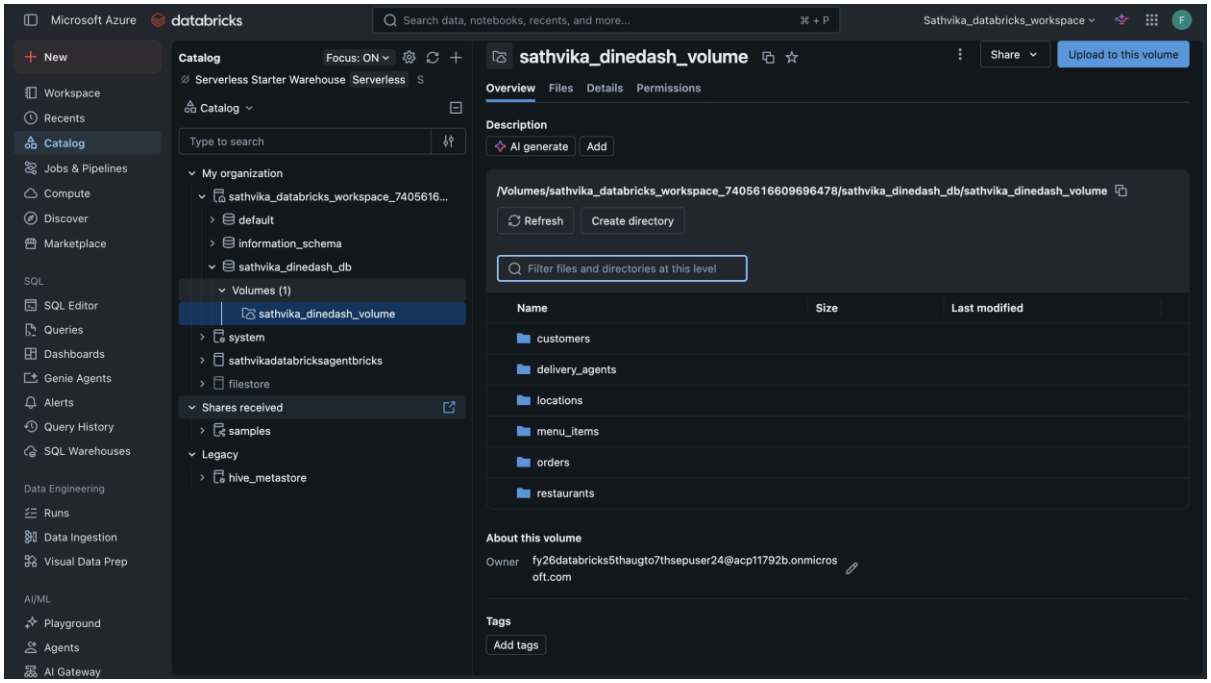


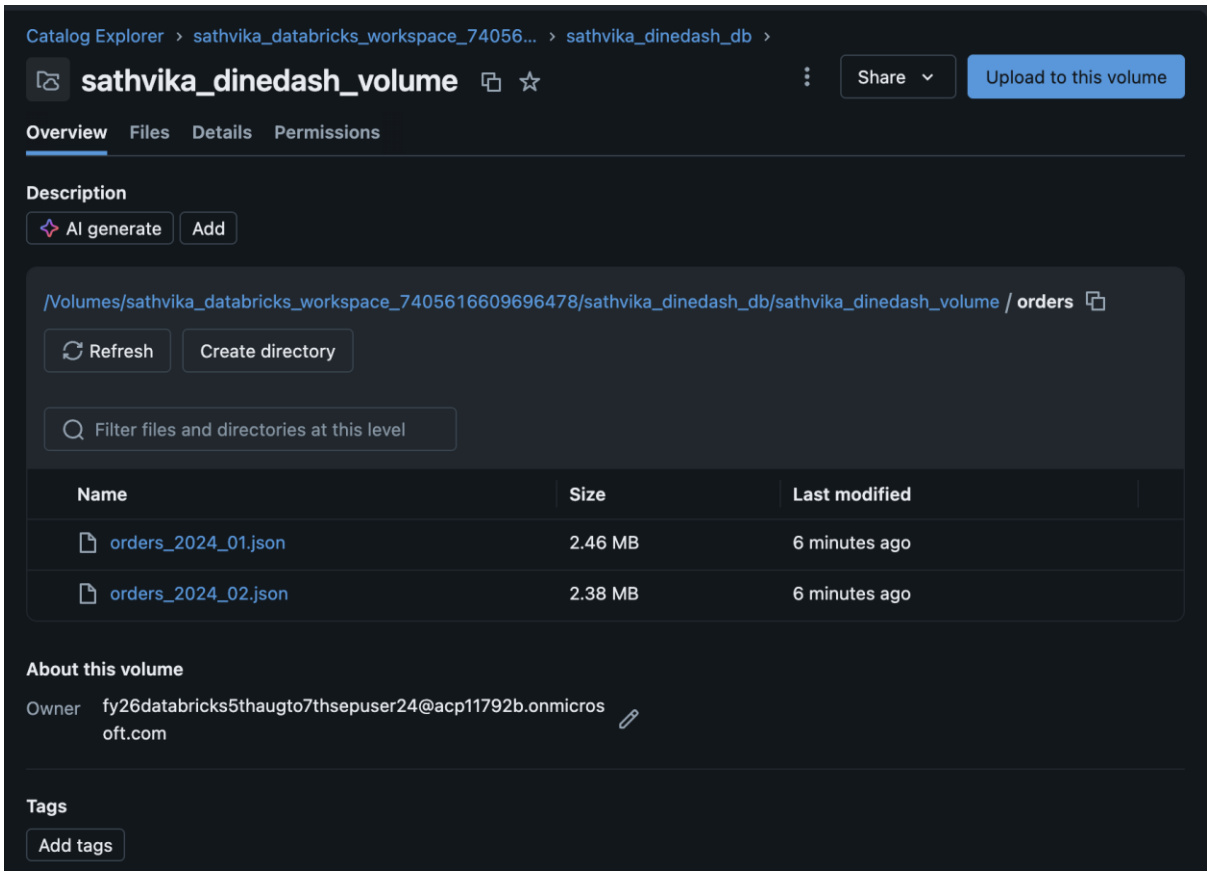
Other datasets files:



DATA INGESTION

The DineDash project begins with ingesting data from multiple source datasets into the Databricks environment. The source data consists of monthly JSON order files and dimension datasets such as customers, restaurants, locations, delivery agents, and menu items. For this implementation, the datasets are organized in a managed Databricks Volume under separate directories for each entity, namely orders, customers, restaurants, locations, delivery_agents, and menu_items. Initially, the orders directory contains only the January and February order JSON files, while the corresponding dimension directories contain their respective CSV files. This organization makes the source data easier to manage and allows additional monthly order files to be added incrementally later. The ingested data is then used as the source for the Bronze layer, followed by Silver and Gold transformations as part of the DineDash ETL pipeline.

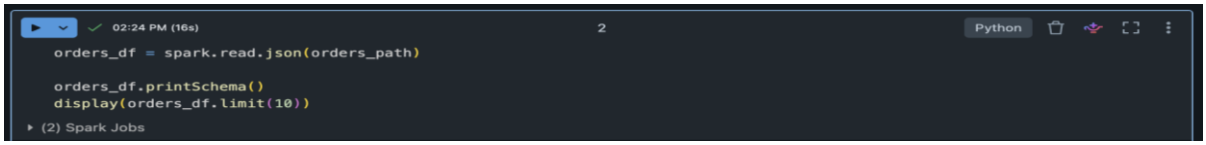


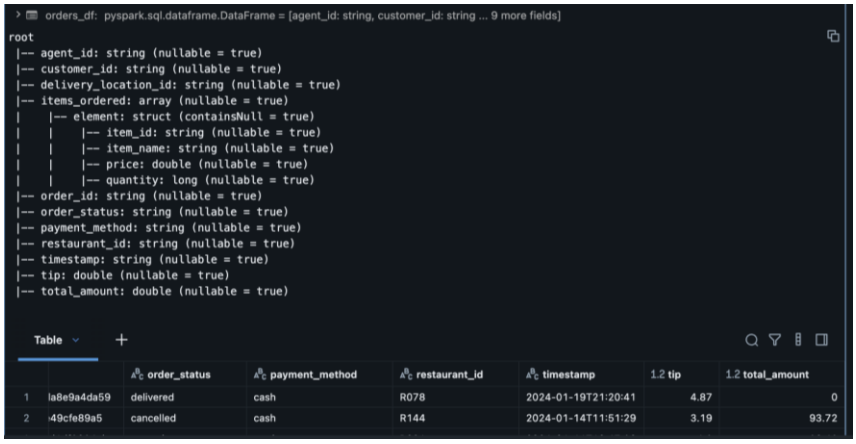


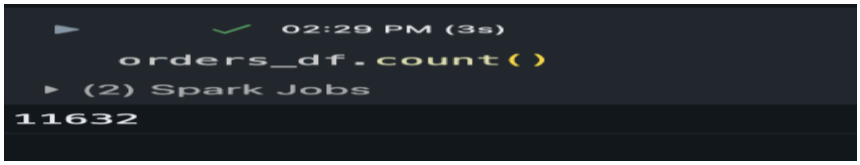
Volume path:
"/Volumes/sathvika_databricks_workspace_7405616609696478/sathvika_dinedash_db/sathvika_dinedash_volume/”

Figure : **DineDash managed volume structure with separate directories**

Created a Spark DataFrame, and displays the first 10 customer records,similarly for all the tables .









Initial Data Exploration

Querying the files directly

Explore each of the dataset by querying against the DBFS path of each dataset files directly. Observe the sample records to understand about the dataset by displaying all the columns for 10 records.

Output:

Customer's Table:



Table

	customer_id	name	email	dob	signup_date	location_id
2	C0002	Justin Diaz	millernathan@example.net	1985-07-05	2018-09-06	L016
3	C0003	Patricia Pena	kristinesanchez@example.com	2003-10-19	2012-05-21	L049
4	C0004	David Smith	cassandra07@example.org	1993-02-22	2003-03-29	L011
5	C0005	Meghan George	johnsonrandall@example.com	1956-05-17	2016-01-27	L071
6	C0006	Jennifer Beard	djohnson@example.net	1974-02-12	1991-01-17	L038
7	C0007	Tiffany Davis	thomasjoseph@example.com	1978-05-20	1998-04-16	L081
8	C0008	Rachel Elliott	pbarnes@example.com	1964-07-20	1997-04-21	L080
9	C0009	Mark Martin	ijohnson@example.org	1964-05-01	1980-04-16	L047
10	C0010	Julie Murphy	amy49@example.net	1968-05-28	2014-11-27	L074
11	C0011	Kelli Luna	hdean@example.net	1954-10-28	1970-05-11	L025
12	C0012	Keith Adams	stephenwilliamson@example.net	1983-10-04	1992-11-11	L091
13	C0013	Toni Barber	natasha96@example.com	1959-09-24	2007-08-16	L009
14	C0014	William Galloway	gray@example.org	1961-08-17	1969-04-15	L006
15	C0015	Suzanne Martinez	emily87@example.net	1980-04-08	2009-12-02	L085
16	C0016	Virginia Molina	nielsenbradley@example.com	1991-06-28	2005-10-12	L030

10,000+ rows | Truncated data | 2.210s runtime | Refreshed Now

Restaurant's Table:

	restaurant_id	name	cuisines	location_id	rating	delivery_fee
1	R001	Food Haven Grill	Indian	L069	3.3	3.9
2	R002	Spice Villa Hub	Thai,Italian,Chinese	L012	3.8	2.25
3	R003	null	Indian,Thai	L076	0.0	0.0
4	R004	Curry Leaf Hub	Indian,American	L048	3.1	3.03
5	R005	Tasty Bites Grill	Mexican	L039	3.9	1.02
6	R006	null	Thai,American	L077	0.0	0.0
7	R007	Pasta Palace Corner	American,Mexican,Chinese	L014	3.3	3.33
8	R008	Curry Leaf Grill	Chinese,Mexican	L053	2.7	1.25
9	R009	Curry Leaf Corner	Mexican	L010	3.4	3.11
10	R010	Pasta Palace Hub	Mexican,Italian,American	L082	3.6	2.76

Menu Items Table:

	item_id	restaurant_id	item_name	price	category	_rescued_data
1	I0001	R017	Radio Soda	17.97	Beverage	null
2	I0002	R169	Occur Pizza	27.44	Main Course	null
3	I0003	R178	Road Nachos	9.5	Appetizer	null
4	I0004	R032	Detail Juice	5.98	Beverage	null
5	I0005	R042	Floor Nachos	7.2	Appetizer	null
6	I0006	R022	Benefit Ice Cream	29.5	Dessert	null
7	I0007	R140	Firm Cake	9.29	Dessert	null
8	I0008	R077	East Pizza	6.15	Main Course	null
9	I0009	R137	Same Brownie	16.12	Dessert	null
10	I0010	R134	Wrong Juice	0	Beverage	null

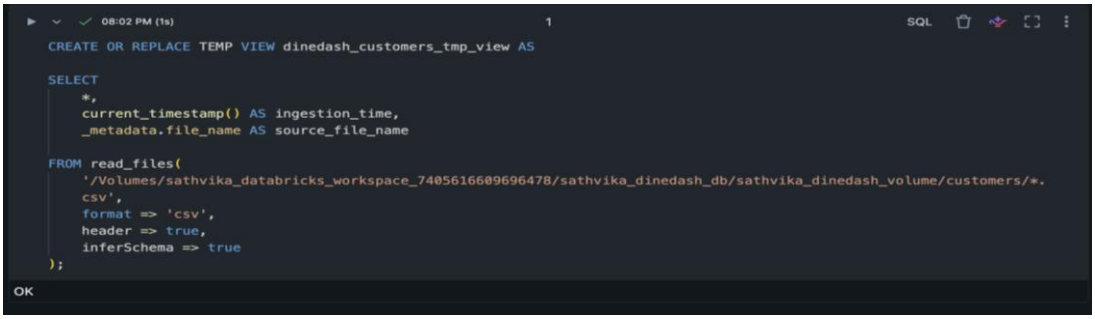
Delivery Agents Table:

	agent_id	name	phone	rating	_rescued_data
1	A001	Lindsey Lang	649.286.1390	3.67	null
2	A002	Heidi Rodriguez	597.686.1508x2345	3.87	null
3	A003	Timothy Dudley	306.628.8953x804	4.57	null
4	A004	Victor Caldwell	null	0	null
5	A005	Rebecca Mullen	998-623-6945x94755	4.47	null
6	A006	William Sullivan	(430)391-1726	3.7	null
7	A007	Courtney Stevens	+1-715-785-0893	4.39	null
8	A008	Amy Patterson	(294)393-3957	4.12	null
9	A009	John Pham	001-319-364-9142x33514	4.91	null
10	A010	Sarah Melendez	444.257.2588	4.18	null

Creating Tables

Create temporary views using the **read_files()** table-valued function by inferring schema from the data for each of the datasets for customers, restaurants, locations, delivery_agents, menu_items as:

- dinedash_customers_tmp_view



- dinedash_restaurants_tmp_view



- dinedash_locations_tmp_view

```
▶  ✓ 08:11 PM (1s)                                                    3

CREATE OR REPLACE TEMP VIEW dinedash_locations_tmp_view
AS
SELECT *,current_timestamp() AS ingestion_time,_metadata.file_name AS source_file_name
FROM read_files(
  '/Volumes/sathvika_databricks_workspace_7405616609696478/sathvika_dinedash_db/sathvika_dinedash_volume/locations/*.csv',
  format => 'csv',
  header => true,
  inferSchema => true
);

▸ (1) Spark Jobs
OK
```

- dinedash_delivery_agents_tmp_view

```
▶  ✓ 08:05 PM (1s)                                                    2

CREATE OR REPLACE TEMP VIEW dinedash_delivery_agents_tmp_view
AS
SELECT *,current_timestamp() AS ingestion_time,_metadata.file_name AS source_file_name
FROM read_files(
  '/Volumes/sathvika_databricks_workspace_7405616609696478/sathvika_dinedash_db/sathvika_dinedash_volume/delivery_agents/
  *.csv',
  format => 'csv',
  header => true,
  inferSchema => true
);

▸ (1) Spark Jobs
```

- dinedash_menu_items_tmp_view

```
▶  ✓ 08:15 PM (1s)                                                    4                                                SQL

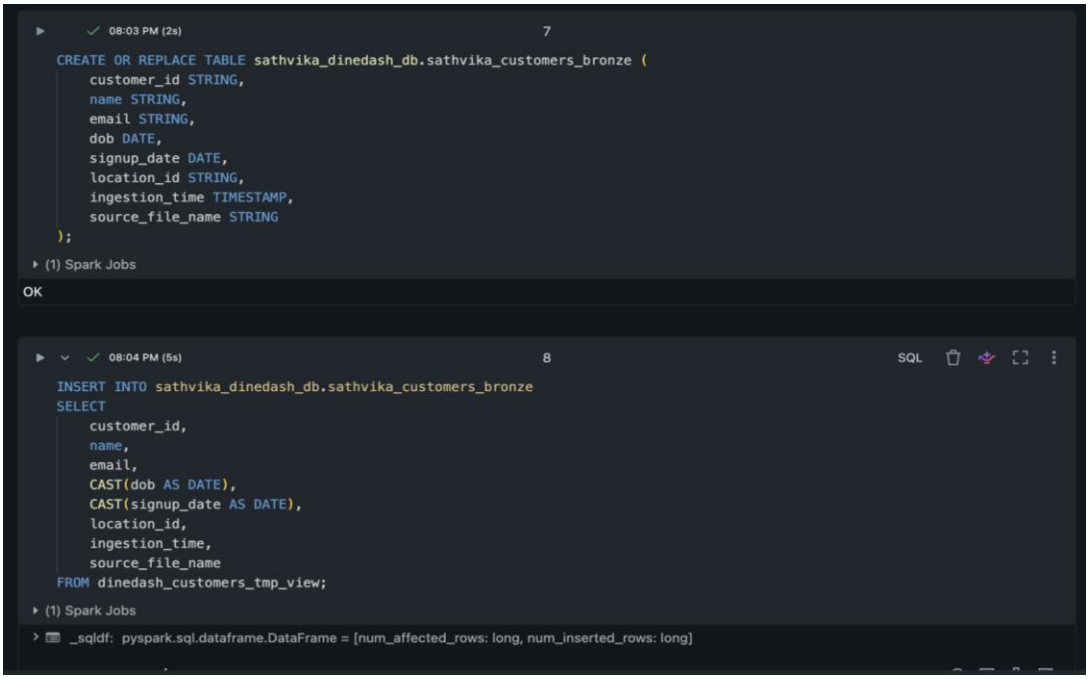
CREATE OR REPLACE TEMP VIEW dinedash_menu_items_tmp_view
AS
SELECT *,current_timestamp() AS ingestion_time, _metadata.file_name AS source_file_name
FROM read_files(
  '/Volumes/sathvika_databricks_workspace_7405616609696478/sathvika_dinedash_db/sathvika_dinedash_volume/menu_items/*.
  csv',
  format => 'csv',
  header => true,
  inferSchema => true
);

▸ (1) Spark Jobs
OK
```

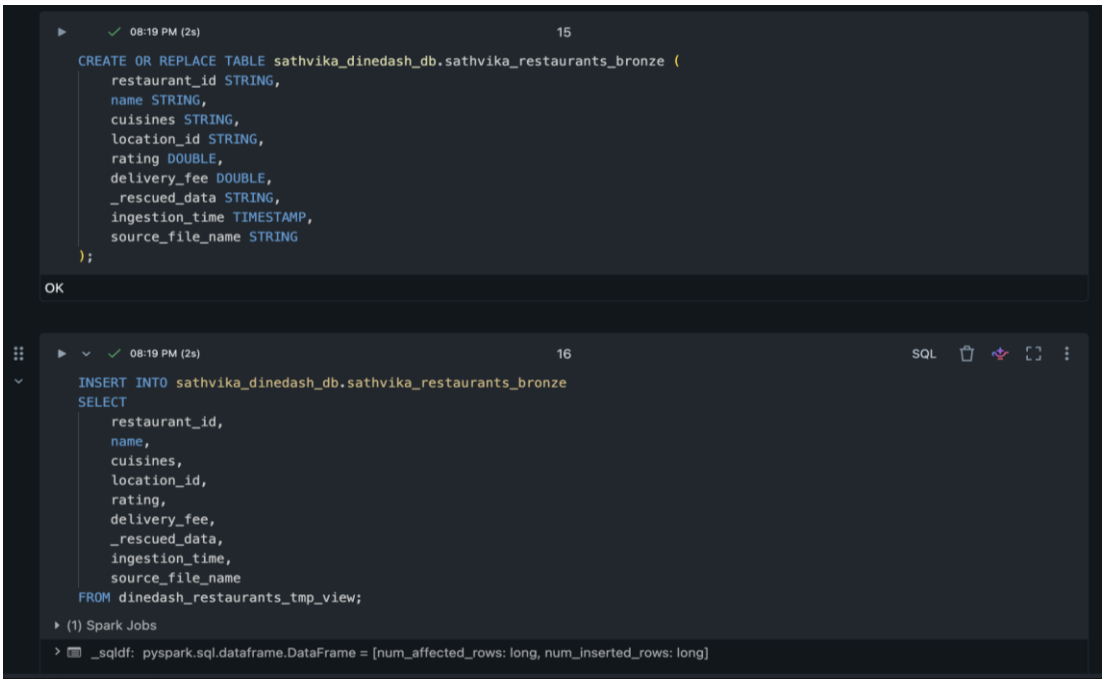
2. Create Raw Delta Tables

The following raw Delta tables were created using manually defined schemas. Each table captures the **record ingestion time** and **source file name** for data lineage and traceability.

- sathvika_customers_bronze



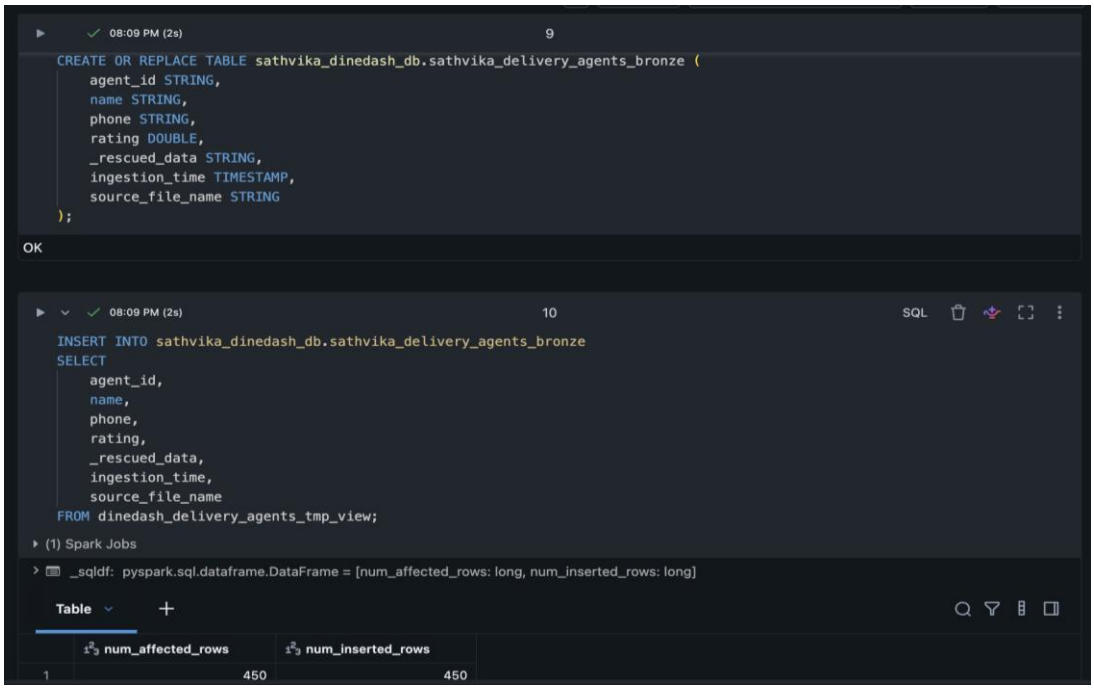
- sathvika_restaurants_bronze



- sathvika_locations_bronze



- sathvika_delivery_agents_bronze



- sathvika_menu_items_bronze

```
08:17 PM (2s)                                                    13
CREATE OR REPLACE TABLE sathvika_dinedash_db.sathvika_menu_items_bronze (
    item_id STRING,
    restaurant_id STRING,
    item_name STRING,
    price DOUBLE,
    category STRING,
    _rescued_data STRING,
    ingestion_time TIMESTAMP,
    source_file_name STRING
);
OK

08:17 PM (2s)                                                    14                SQL
INSERT INTO sathvika_dinedash_db.sathvika_menu_items_bronze
SELECT
    item_id,
    restaurant_id,
    item_name,
    price,
    category,
    _rescued_data,
    ingestion_time,
    source_file_name
FROM dinedash_menu_items_tmp_view;
▸ (1) Spark Jobs
▸ _sqldf: pyspark.sql.dataframe.DataFrame = [num_affected_rows: long, num_inserted_rows: long]
```

sathvika_customers_bronze

```
08:24 PM (<1s)                                                   17                SQL
DESCRIBE sathvika_dinedash_db.sathvika_customers_bronze;
▸ _sqldf: pyspark.sql.dataframe.DataFrame = [col_name: string, data_type: string ... 1 more field]
```

Table

	col_name	data_type	comment
1	customer_id	string	null
2	name	string	null
3	email	string	null
4	dob	date	null
5	signup_date	date	null
6	location_id	string	null
7	ingestion_time	timestamp	null
8	source_file_name	string	null

8 rows | 0.395s runtime — Refreshed 5m ago

The sathvika_customers_bronze table stores raw customer data along with ingestion timestamp and source file name for data lineage and traceability.

	email	dob	signup_date	location_id	ingestion_time	source_file_nar
1	heatherdouglas@example.org	1954-12-02	2019-05-16	L069	2026-08-22T08:08:22.357+00:00	dim_customers.csv
2	millernathan@example.net	1985-07-05	2018-09-06	L016	2026-08-22T08:08:22.357+00:00	dim_customers.csv
3	kristinesanchez@example.co...	2003-10-19	2012-05-21	L049	2026-08-22T08:08:22.357+00:00	dim_customers.csv
4	cassandra07@example.org	1993-02-22	2003-03-29	L011	2026-08-22T08:08:22.357+00:00	dim_customers.csv
5	johnsonrandall@example.com	1956-05-17	2016-01-27	L071	2026-08-22T08:08:22.357+00:00	dim_customers.csv
6	djohnson@example.net	1974-02-12	1991-01-17	L038	2026-08-22T08:08:22.357+00:00	dim_customers.csv
7	thomasjoseph@example.com	1978-05-20	1998-04-16	L081	2026-08-22T08:08:22.357+00:00	dim_customers.csv
8	pbarnes@example.com	1964-07-20	1997-04-21	L080	2026-08-22T08:08:22.357+00:00	dim_customers.csv
9	ijohnson@example.org	1964-05-01	1980-04-16	L047	2026-08-22T08:08:22.357+00:00	dim_customers.csv

sathvika_restaurants_bronze

	col_name	data_type	comment
1	restaurant_id	string	null
2	name	string	null
3	cuisines	string	null
4	location_id	string	null
5	rating	double	null
6	delivery_fee	double	null
7	_rescued_data	string	null
8	ingestion_time	timestamp	null
9	source_file_name	string	null

The sathvika_restaurants_bronze table stores raw restaurant details along with ingestion timestamp and source file name for data lineage and traceability.

location_id	rating	delivery_fee	_rescued_data	ingestion_time	source_file_name
L069	3.3	3.9	null	2026-08-22T08:18:39.910+00:00	dim_restaurants.csv
L012	3.8	2.25	null	2026-08-22T08:18:39.910+00:00	dim_restaurants.csv
L076	0	0	null	2026-08-22T08:18:39.910+00:00	dim_restaurants.csv
L048	3.1	3.03	null	2026-08-22T08:18:39.910+00:00	dim_restaurants.csv
L039	3.9	1.02	null	2026-08-22T08:18:39.910+00:00	dim_restaurants.csv
L077	0	0	null	2026-08-22T08:18:39.910+00:00	dim_restaurants.csv
L014	3.3	3.33	null	2026-08-22T08:18:39.910+00:00	dim_restaurants.csv
L053	2.7	1.25	null	2026-08-22T08:18:39.910+00:00	dim_restaurants.csv
L010	3.4	3.11	null	2026-08-22T08:18:39.910+00:00	dim_restaurants.csv

sathvika_locations_bronze

	col_name	data_type	comment
1	location_id	string	null
2	area	string	null
3	city	string	null
4	state	string	null
5	latitude	double	null
6	longitude	double	null
7	_rescued_data	string	null
8	ingestion_time	timestamp	null
9	source_file_name	string	null

The sathvika_locations_bronze table stores raw location details including state, latitude, and longitude, along with ingestion timestamp and source file name for data lineage.

	state	latitude	longitude	_rescued_data	ingestion_time	source_file_name
1	Michigan	-2.8767365	-62.888285	null	2026-08-22T08:22:52.945+00:00	dim_locations.csv
2	Washington	-54.1725655	-46.757126	null	2026-08-22T08:22:52.945+00:00	dim_locations.csv
3	Michigan	-19.072773	27.938702	null	2026-08-22T08:22:52.945+00:00	dim_locations.csv
4	Texas	13.0952805	141.629215	null	2026-08-22T08:22:52.945+00:00	dim_locations.csv
5	Wisconsin	8.239121	26.958565	null	2026-08-22T08:22:52.945+00:00	dim_locations.csv
6	Missouri	16.931649	-49.121678	null	2026-08-22T08:22:52.945+00:00	dim_locations.csv
7	West Virginia	-66.185224	-142.948244	null	2026-08-22T08:22:52.945+00:00	dim_locations.csv
8	Oklahoma	-83.706852	28.380354	null	2026-08-22T08:22:52.945+00:00	dim_locations.csv
9	Maryland	15.1168605	-67.825285	null	2026-08-22T08:22:52.945+00:00	dim_locations.csv
10	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]

sathvika_delivery_agents_bronze

	col_name	data_type	comment
1	agent_id	string	null
2	name	string	null
3	phone	string	null
4	rating	double	null
5	_rescued_data	string	null
6	ingestion_time	timestamp	null
7	source_file_name	string	null

The sathvika_delivery_agents_bronze table stores raw delivery agent details including name, phone, and rating, along with ingestion timestamp and source file name for data lineage.

	name	phone	rating	_rescued_data	ingestion_time	source_file_name
1	ndsey Lang	649.286.1390	3.67	null	2026-08-22T08:26:36.030+00:00	dim_delivery_agents.csv
2	eidi Rodriguez	597.686.1508x2345	3.87	null	2026-08-22T08:26:36.030+00:00	dim_delivery_agents.csv
3	mothy Dudley	306.628.8953x804	4.57	null	2026-08-22T08:26:36.030+00:00	dim_delivery_agents.csv
4	ctor Caldwell	null	0	null	2026-08-22T08:26:36.030+00:00	dim_delivery_agents.csv
5	ebecca Mullen	998-623-6945x94755	4.47	null	2026-08-22T08:26:36.030+00:00	dim_delivery_agents.csv
6	illiam Sullivan	(430)391-1726	3.7	null	2026-08-22T08:26:36.030+00:00	dim_delivery_agents.csv
7	urtney Stevens	+1-715-785-0893	4.39	null	2026-08-22T08:26:36.030+00:00	dim_delivery_agents.csv
8	ny Patterson	(294)393-3957	4.12	null	2026-08-22T08:26:36.030+00:00	dim_delivery_agents.csv
9	hn Pham	001-319-364-9142x33514	4.91	null	2026-08-22T08:26:36.030+00:00	dim_delivery_agents.csv

sathvika_menu_items_bronze

	col_name	data_type	comment
1	item_id	string	null
2	restaurant_id	string	null
3	item_name	string	null
4	price	double	null
5	category	string	null
6	_rescued_data	string	null
7	ingestion_time	timestamp	null
8	source_file_name	string	null

The sathvika_menu_items_bronze table stores raw menu item details including item name, price, and category, along with ingestion timestamp and source file name for data lineage.

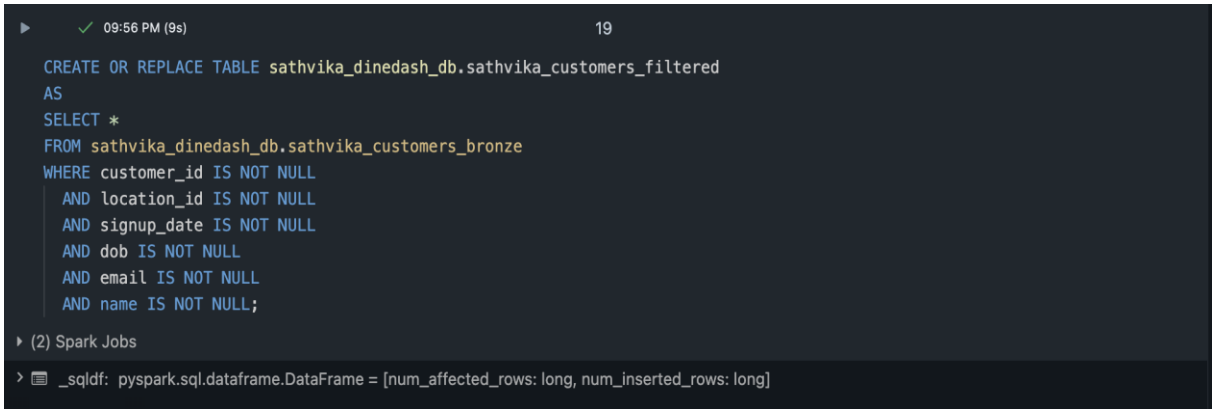
	item_name	price	category	_rescued_data	ingestion_time	source_file_name
1	Radio Soda	17.97	Beverage	null	2026-08-22T08:31:35.584+00:00	dim_menu_items.csv
2	Occur Pizza	27.44	Main Course	null	2026-08-22T08:31:35.584+00:00	dim_menu_items.csv
3	Road Nachos	9.5	Appetizer	null	2026-08-22T08:31:35.584+00:00	dim_menu_items.csv
4	Detail Juice	5.98	Beverage	null	2026-08-22T08:31:35.584+00:00	dim_menu_items.csv
5	Floor Nachos	7.2	Appetizer	null	2026-08-22T08:31:35.584+00:00	dim_menu_items.csv
6	Benefit Ice Cream	29.5	Dessert	null	2026-08-22T08:31:35.584+00:00	dim_menu_items.csv
7	Firm Cake	9.29	Dessert	null	2026-08-22T08:31:35.584+00:00	dim_menu_items.csv
8	East Pizza	6.15	Main Course	null	2026-08-22T08:31:35.584+00:00	dim_menu_items.csv
9	Same Brownie	16.12	Dessert	null	2026-08-22T08:31:35.584+00:00	dim_menu_items.csv

3. Create cleaned data delta tables

Create following tables from the corresponding bronze tables

sathvika_customers_filtered

Which contains the records where customer_id, location_id, signup_date, dob, email, name are not missing

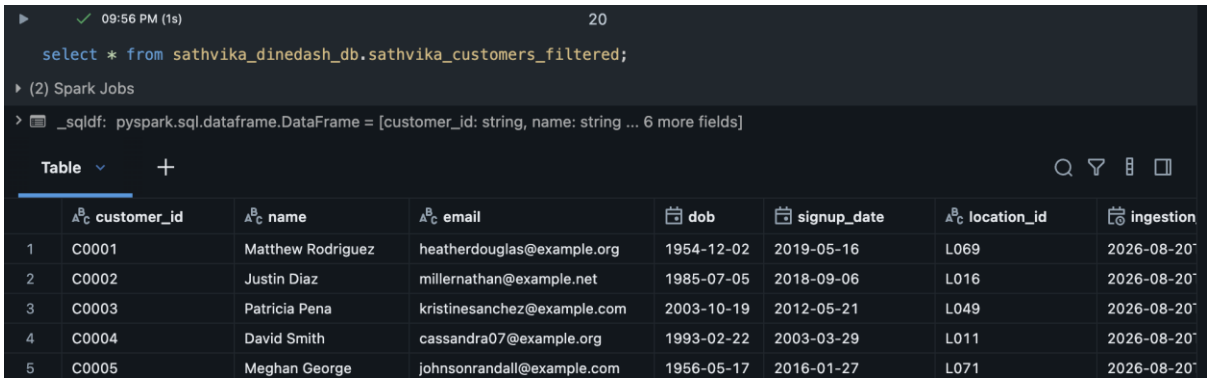


```
09:56 PM (9s)                                                    19

CREATE OR REPLACE TABLE sathvika_dinedash_db.sathvika_customers_filtered
AS
SELECT *
FROM sathvika_dinedash_db.sathvika_customers_bronze
WHERE customer_id IS NOT NULL
  AND location_id IS NOT NULL
  AND signup_date IS NOT NULL
  AND dob IS NOT NULL
  AND email IS NOT NULL
  AND name IS NOT NULL;

(2) Spark Jobs

_sqldf:  pyspark.sql.dataframe.DataFrame = [num_affected_rows: long, num_inserted_rows: long]
```



```
09:56 PM (1s)                                                    20

select * from sathvika_dinedash_db.sathvika_customers_filtered;

(2) Spark Jobs

_sqldf:  pyspark.sql.dataframe.DataFrame = [customer_id: string, name: string ... 6 more fields]
```

Table

	customer_id	name	email	dob	signup_date	location_id	ingestion
1	C0001	Matthew Rodriguez	heatherdouglas@example.org	1954-12-02	2019-05-16	L069	2026-08-20
2	C0002	Justin Diaz	millernathan@example.net	1985-07-05	2018-09-06	L016	2026-08-20
3	C0003	Patricia Pena	kristinesanchez@example.com	2003-10-19	2012-05-21	L049	2026-08-20
4	C0004	David Smith	cassandra07@example.org	1993-02-22	2003-03-29	L011	2026-08-20
5	C0005	Meghan George	johnsonrandall@example.com	1956-05-17	2016-01-27	L071	2026-08-20

sathvika_restaurants_filtered

Which contains the records where restaurant_id, name, cuisines, location_id, rating, delivery_fee are not missing



```
09:58 PM (3s)                                                    21

CREATE OR REPLACE TABLE sathvika_dinedash_db.sathvika_restaurants_filtered
AS
SELECT *
FROM sathvika_dinedash_db.sathvika_restaurants_bronze
WHERE restaurant_id IS NOT NULL
  AND name IS NOT NULL
  AND cuisines IS NOT NULL
  AND location_id IS NOT NULL
  AND rating IS NOT NULL
  AND delivery_fee IS NOT NULL;

(2) Spark Jobs

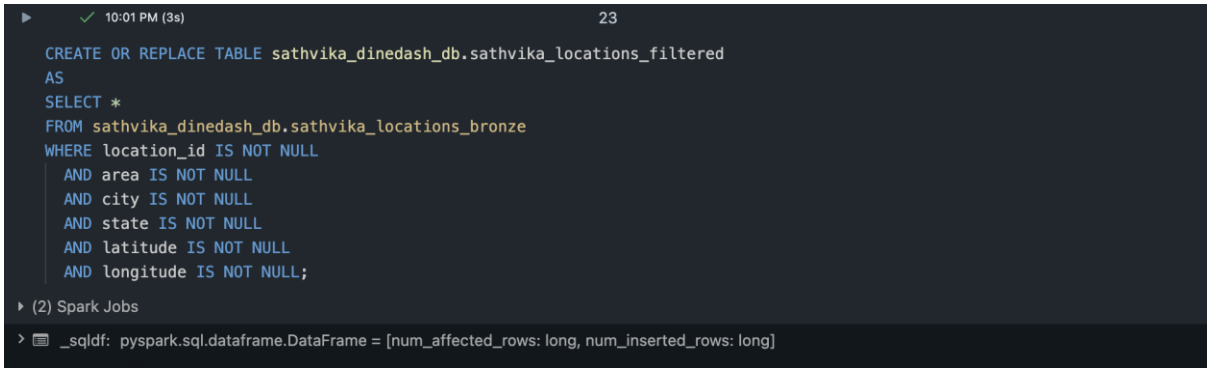
_sqldf:  pyspark.sql.dataframe.DataFrame = [num_affected_rows: long, num_inserted_rows: long]
```

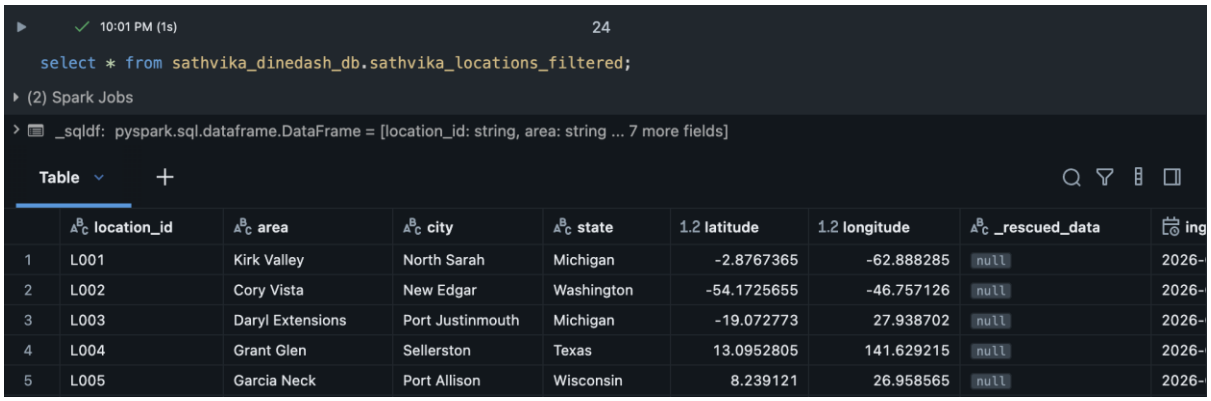


	restaurant_id	name	cuisines	location_id	rating	delivery_fee	_rescued_da
1	R001	Food Haven Grill	Indian	L069	3.3	3.9	null
2	R002	Spice Villa Hub	Thai,Italian,Chinese	L012	3.8	2.25	null
3	R004	Curry Leaf Hub	Indian,American	L048	3.1	3.03	null
4	R005	Tasty Bites Grill	Mexican	L039	3.9	1.02	null
5	R007	Pasta Palace Corner	American,Mexican,Chinese	L014	3.3	3.33	null

sathvika_locations_filtered

Which contains the records where location_id, area, city, state, latitude, longitude are not missing

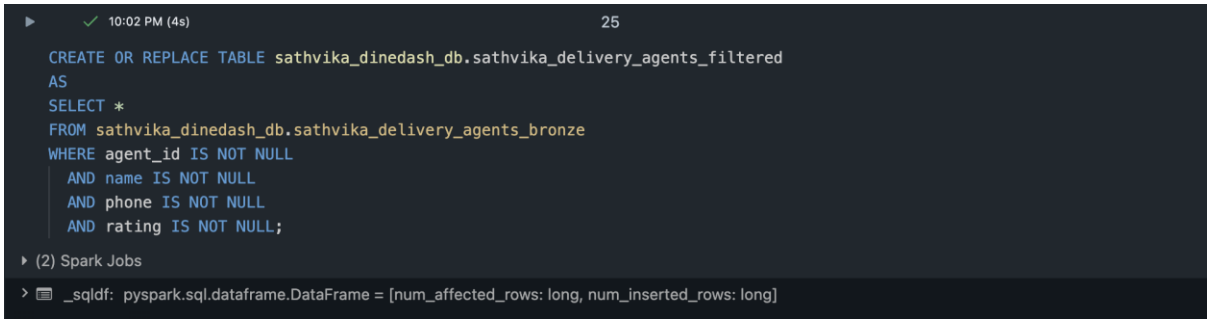


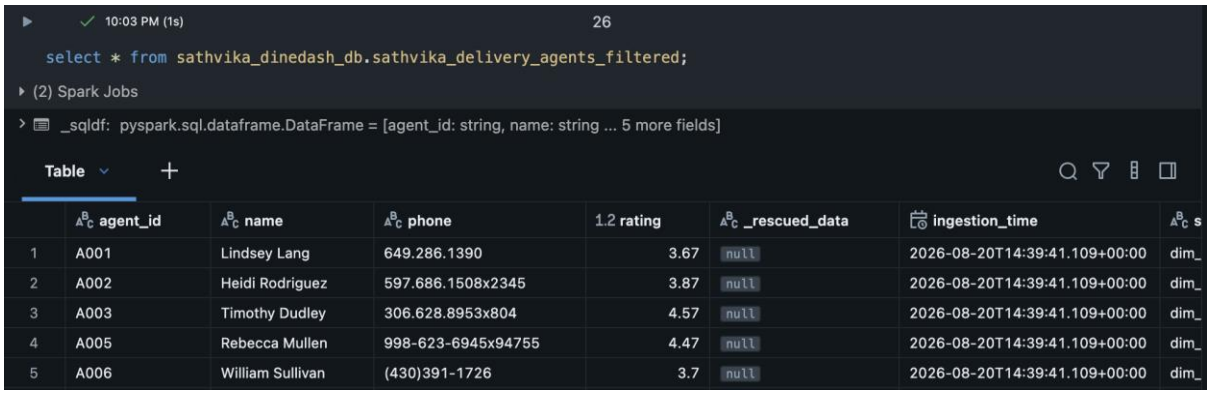


	location_id	area	city	state	latitude	longitude	_rescued_data	ing
1	L001	Kirk Valley	North Sarah	Michigan	-2.8767365	-62.888285	null	2026-
2	L002	Cory Vista	New Edgar	Washington	-54.1725655	-46.757126	null	2026-
3	L003	Daryl Extensions	Port Justinmouth	Michigan	-19.072773	27.938702	null	2026-
4	L004	Grant Glen	Sellerston	Texas	13.0952805	141.629215	null	2026-
5	L005	Garcia Neck	Port Allison	Wisconsin	8.239121	26.958565	null	2026-

sathvika_agents_filtered

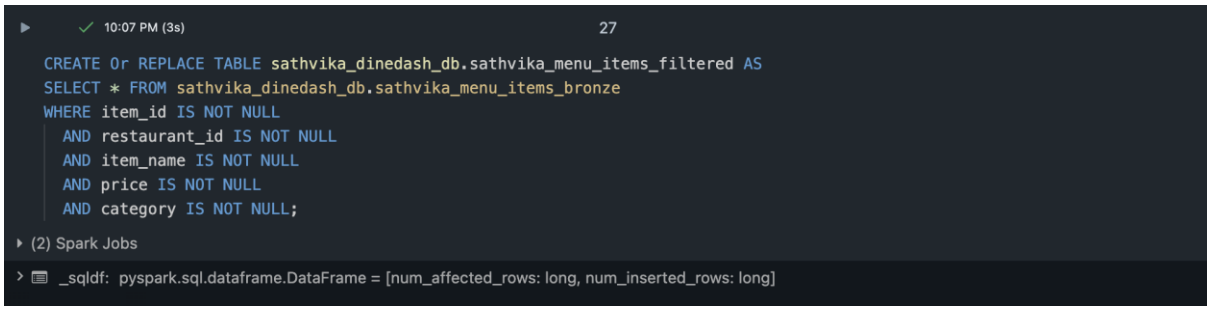
Which contains the records where agent_id, name, phone, rating are not missing





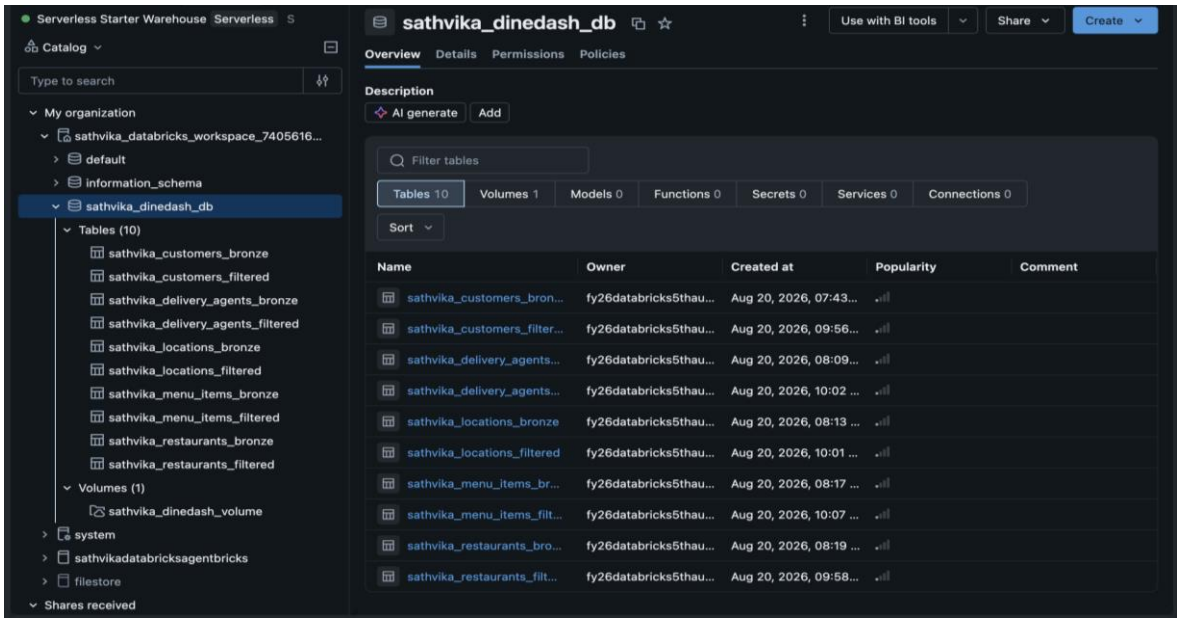
sathvika_menu_items_filtered

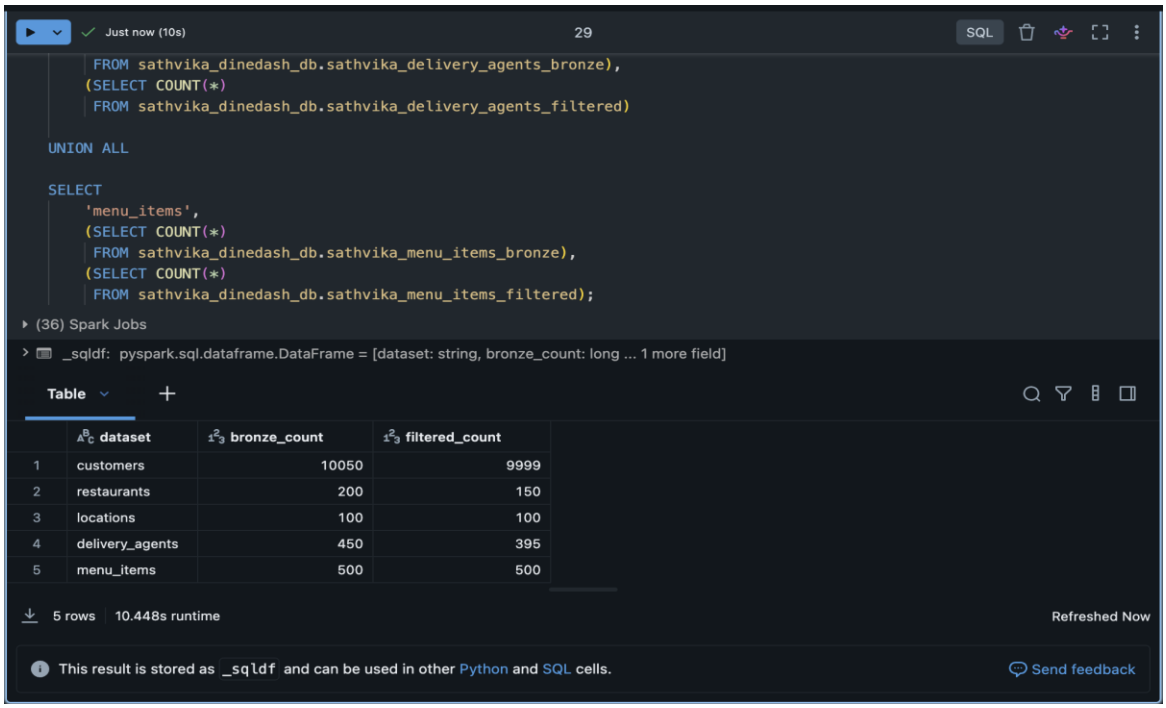
Which contains the records where item_id, restaurant_id, item_name, price, category are not missing





Filtered table validation





Incremental Data Loading

Create a table named **orders_bronze** without schema and incrementally load order details for each month of the year 2024, ingesting one month's data at a time, **using COPY INTO** Ingest January, February month order details, verify

the records count and then ingest March month order details, verify the records count.

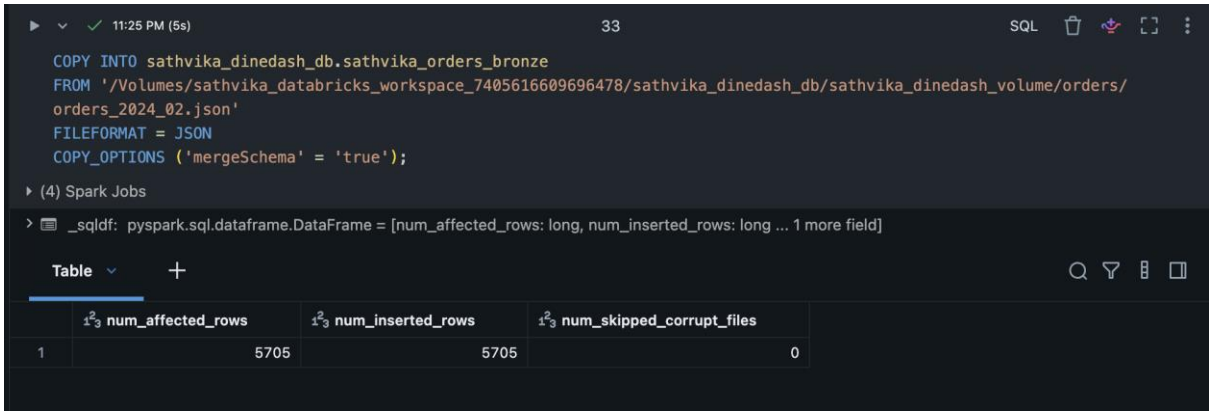
January COPY INTO



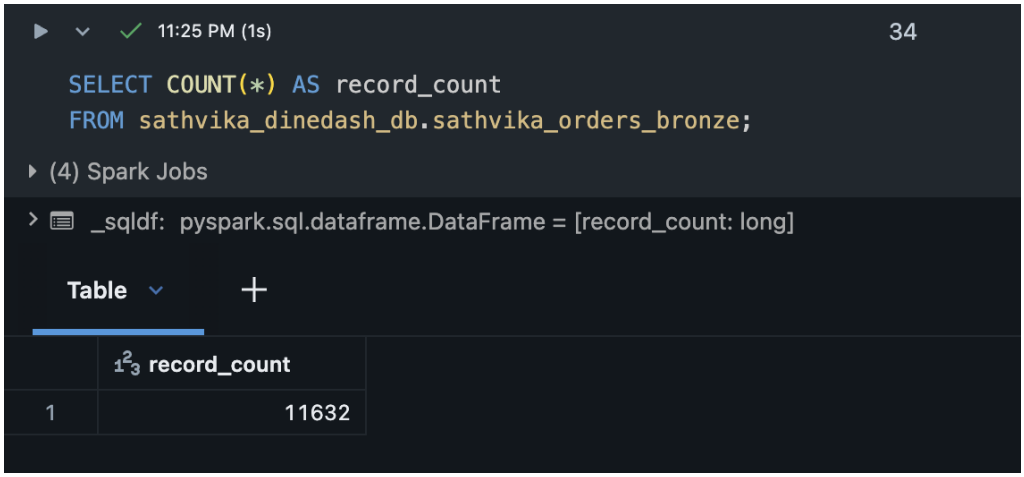
January records count



February COPY INTO



Feb records count



March COPY INTO



March record count



Problem Statements

Note: All numeric calculations should be rounded UP to two decimal places for floating-point numbers and one decimal place for discrete entities. Price mentioned is in dollars ($).

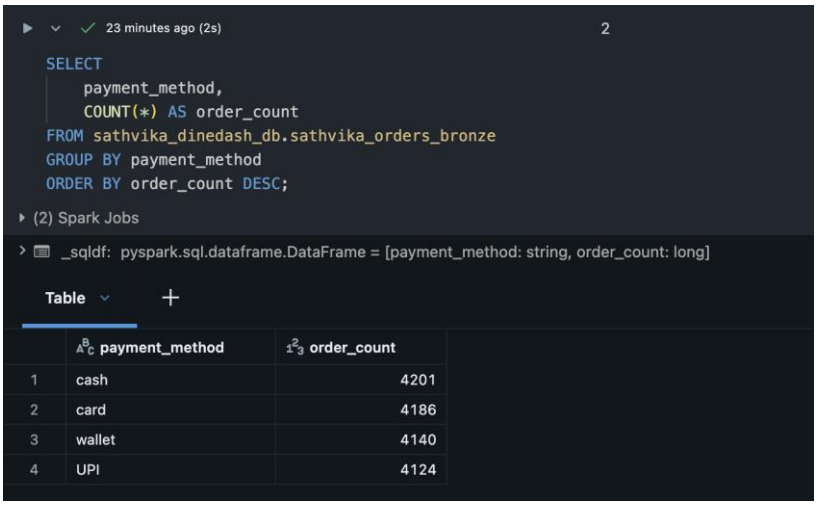
Order Data Analysis

Write Databricks SQL query to find:

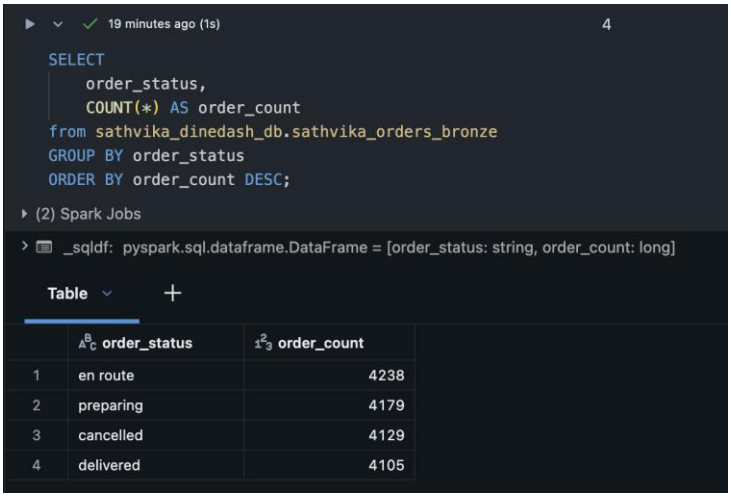
1. Total Orders Placed



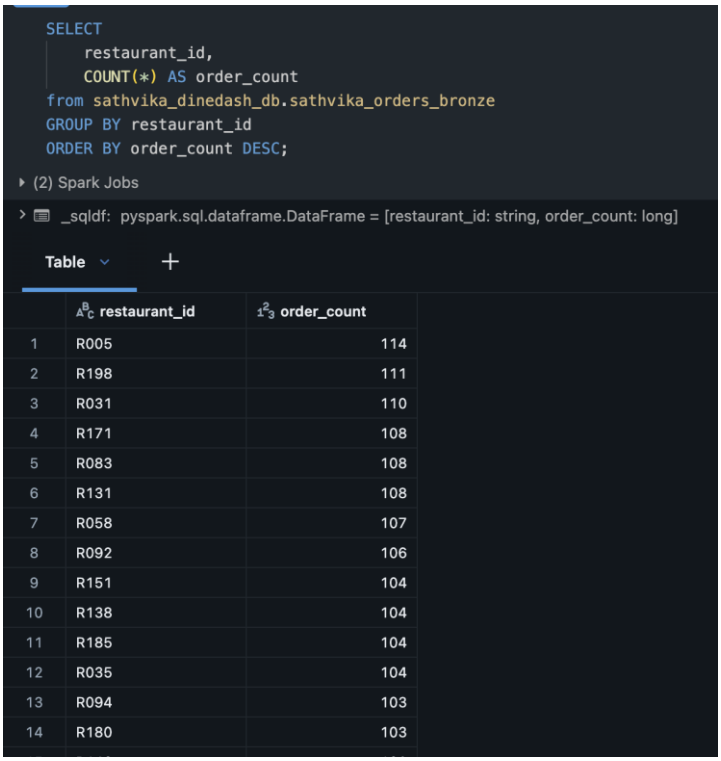
2. Distribution of orders by payment method



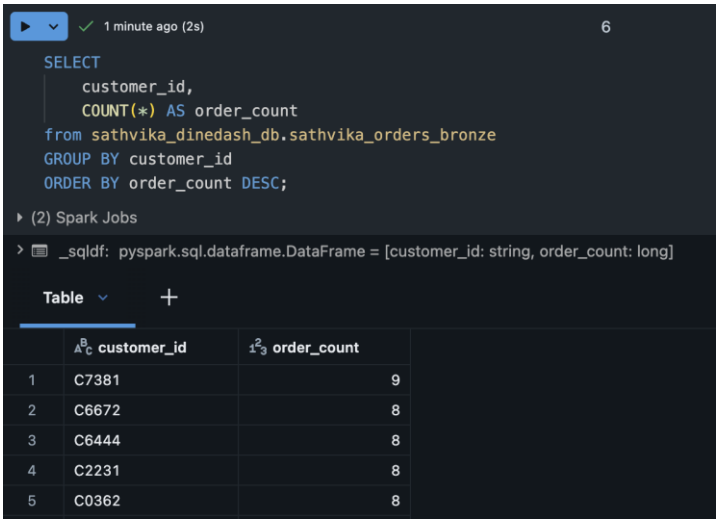
3. Distribution of order status (e.g., delivered, preparing, cancelled)



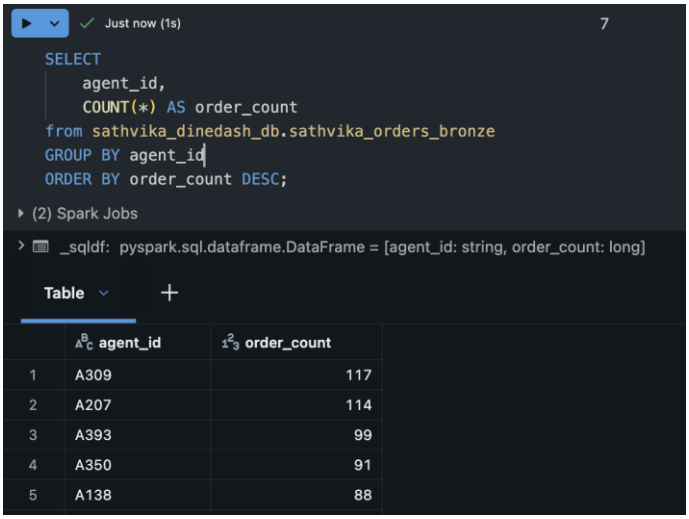
4. Orders per restaurant (volume of business)



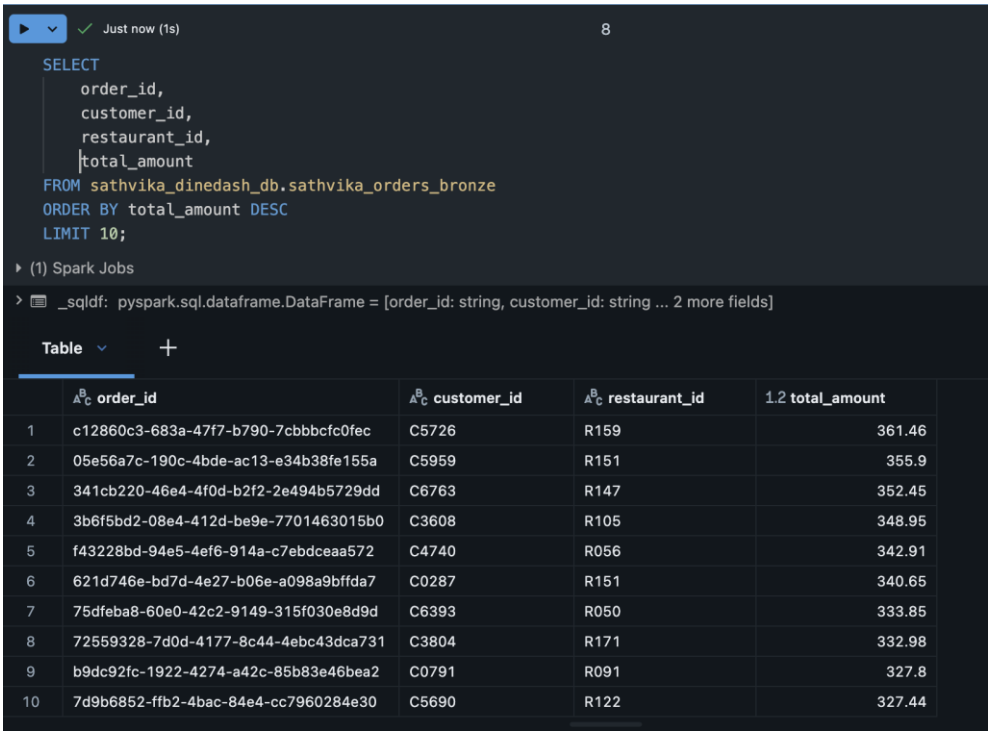
5. Orders per customer (loyalty insight)



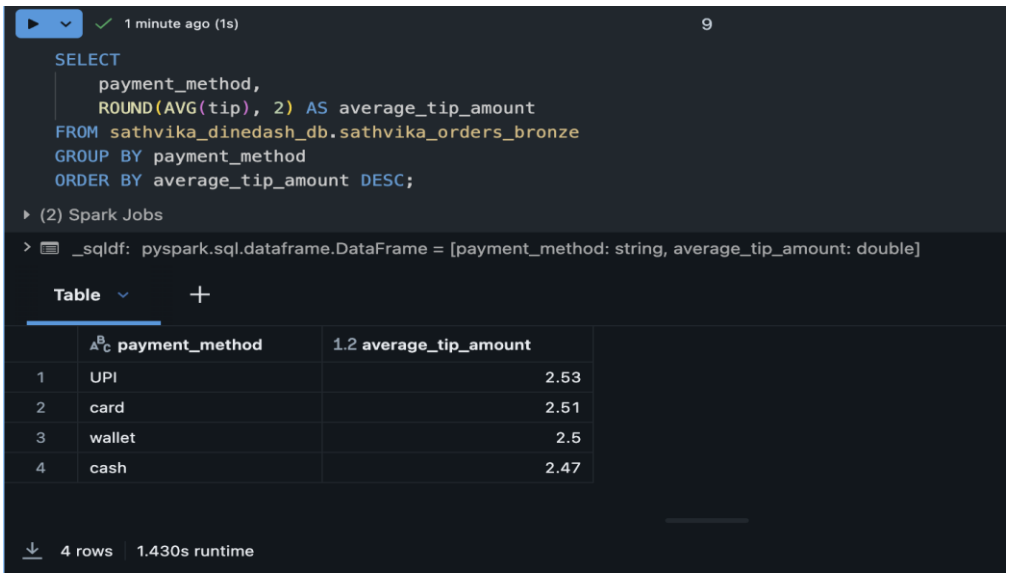
6. Orders handled per delivery agent



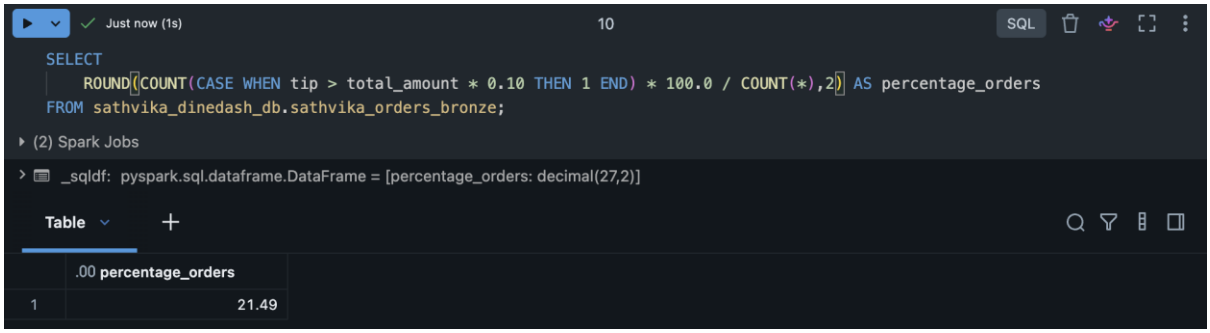
7. Top 10 Orders by total amount



8. Average tip amount per payment method



9. Percentage of Orders with tip > 10% of total_amount



10. Percentage of successful vs cancelled vs failed orders



11. Top Customers (by number of orders) and top restaurants (by number of orders).

Just now (1s) 12

```
SELECT
    customer_id,
    COUNT(*) AS order_count
FROM sathvika_dinedash_db.sathvika_orders_bronze
GROUP BY customer_id
ORDER BY order_count DESC
LIMIT 10;
```

(2) Spark Jobs

_sqldf: pyspark.sql.dataframe.DataFrame = [customer_id: string, order_count: long]

Table

	customer_id	order_count
1	C7381	9
2	C6672	8
3	C6444	8
4	C2231	8
5	C0362	8
6	C1449	8
7	C4347	7
8	C0313	7
9	C2364	7
10	C4867	7

10 rows | 0.876s runtime

Just now (1s) 13

```
SELECT
    restaurant_id,
    COUNT(*) AS order_count
FROM sathvika_dinedash_db.sathvika_orders_bronze
GROUP BY restaurant_id
ORDER BY order_count DESC
LIMIT 10;
```

(2) Spark Jobs

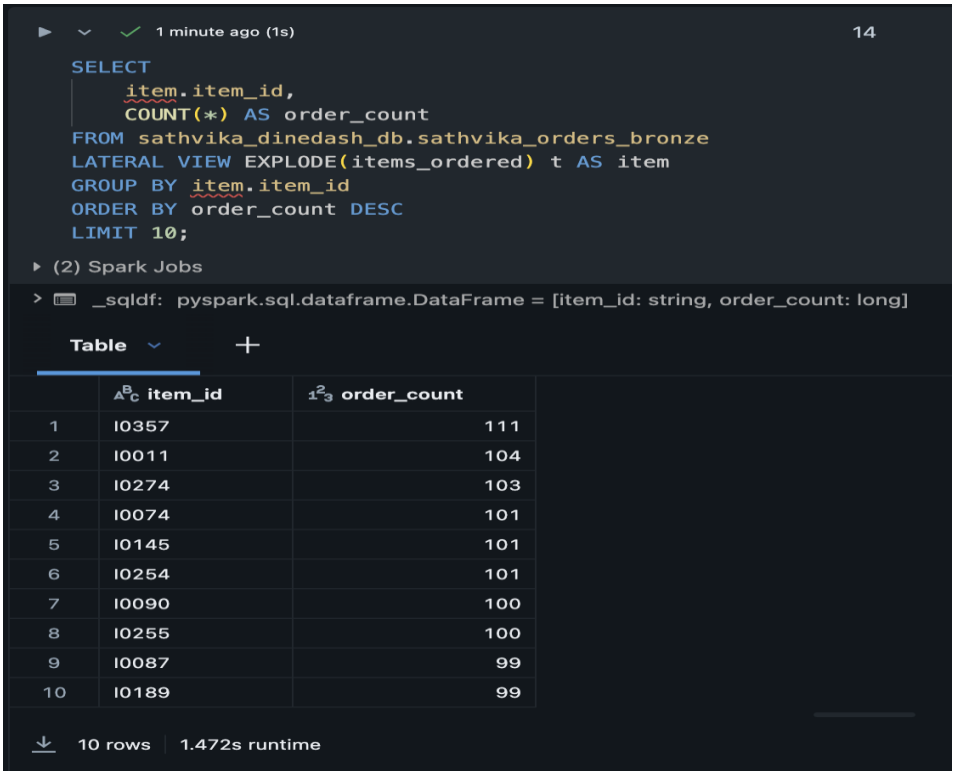
_sqldf: pyspark.sql.dataframe.DataFrame = [restaurant_id: string, order_count: long]

Table

	restaurant_id	order_count
1	R005	114
2	R198	111
3	R031	110
4	R171	108
5	R083	108
6	R131	108
7	R058	107
8	R092	106
9	R138	104
10	R151	104

10 rows | 1.024s runtime

12.a. Find most frequently ordered items



12.b. Find average number of items per order



12.c.Find item revenue contribution per restaurant

```
Just now (1s)                                                    16
SELECT
    restaurant_id,
    item.item_id,
    ROUND(SUM(item.quantity * item.price), 2) AS item_revenue
FROM sathvika_dinedash_db.sathvika_orders_bronze
LATERAL VIEW EXPLODE(items_ordered) t AS item
GROUP BY restaurant_id, item.item_id
ORDER BY item_revenue DESC;
```

▸ (2) Spark Jobs

> _sqldf: pyspark.sql.dataframe.DataFrame = [restaurant_id: string, item_id: string ... 1 mor

Table +

	restaurant_id	item_id	item_revenue
1	R088	I0306	8772.96
2	R039	I0254	8771.2
3	R089	I0500	8534.17
4	R198	I0357	8028.15
5	R102	I0189	7978.38
6	R089	I0108	7636.23
7	R171	I0253	7533.64
8	R084	I0074	7443.14
9	R104	I0284	7396.8
10	R139	I0145	7207.2

Orders Data and Dimension Datasets Analysis

1. Find the preferred restaurant names for each customer.

Code:

```
05:45 PM (1s)
-- 1. Find the preferred restaurant names for each customer
SELECT
    o.customer_id,
    r.name AS restaurant_name,
    COUNT(*) AS order_count
FROM sathvika_dinedash_db.sathvika_orders_bronze o
JOIN sathvika_dinedash_db.sathvika_restaurants_bronze r
    ON o.restaurant_id = r.restaurant_id
GROUP BY
    o.customer_id,
    r.name
ORDER BY
    o.customer_id,
    order_count DESC;
```

▸ (3) Spark Jobs

Ouput:

_sqldf: pyspark.sql.dataframe.DataFrame = [customer_id: string, restaurant_name: string ... 1 more field]

Table +

	customer_id	restaurant_name	order_count
1	C0001	Spice Villa Grill	2
2	C0001	Curry Leaf Hub	1
3	C0002	null	1
4	C0003	Spice Villa Grill	1
5	C0004	Pasta Palace Kitchen	1
6	C0004	Tasty Bites Corner	1
7	C0004	null	1
8	C0006	Pasta Palace Grill	1
9	C0007	Tasty Bites Hub	2
10	C0007	Spice Villa Grill	1

2. Find the top 10 areas receiving orders in each month.

Code:

```
-- 2. Find the top 10 areas receiving orders in each month
SELECT month, area_name, order_count
FROM (
    SELECT
        MONTH(o.timestamp) AS month,
        l.area AS area_name,
        COUNT(*) AS order_count,
        ROW_NUMBER() OVER (
            PARTITION BY MONTH(o.timestamp)
            ORDER BY COUNT(*) DESC
        ) AS rn
    FROM sathvika_dinedash_db.sathvika_orders_bronze o
    JOIN sathvika_dinedash_db.sathvika_locations_bronze l
        ON o.delivery_location_id = l.location_id
    GROUP BY MONTH(o.timestamp), l.area
)
WHERE rn <= 10
ORDER BY month, order_count DESC;
```

Output:

_sqldf: pyspark.sql.dataframe.DataFrame = [month: integer, area_name: string ... 1 more field]

Table

	month	area_name	order_count
1	1	Rodriguez Mill	87
2	1	Eric Hills	86
3	1	Hill Station	80
4	1	Smith Estates	79
5	1	Harper Street	76
6	1	Morgan Key	75
7	1	Copeland Parks	75
8	1	John Underpass	74
9	1	Ashley Extensions	73
10	1	Jennifer Wall	73

3. Find all successfully delivered orders that were paid using a card, to analyze payment preferences and delivery success rates.

Code:

05:48 PM (1s) 3

```
-- 3. Find all successfully delivered orders that were paid using a card, to analyze payment preferences and delivery success rates
SELECT
    order_id,
    customer_id,
    restaurant_id,
    payment_method,
    order_status,
    total_amount
FROM sathvika_dinedash_db.sathvika_orders_bronze
WHERE order_status = 'delivered'
  AND payment_method = 'card';
```

(2) Spark Jobs

Output:

_sqldf: pyspark.sql.dataframe.DataFrame = [order_id: string, customer_id: string ... 4 more fields]

Table

	order_id	customer_id	restaurant_id	payment_method	order_status	total_amount
1	e293e41d-e335-4d6b-8504-34f96140e660	C5605	R006	card	delivered	21.2
2	c8bd7ee7-a451-447a-811b-fd69aff36062	C7525	R053	card	delivered	75.97
3	e71e1b46-c026-4913-9b58-870fde58e72b	C1909	R047	card	delivered	113.32
4	ede3312f-d227-42a5-a1d0-d191999abbd1	C3246	R136	card	delivered	112.3
5	7e5fb9b0-c7e0-4080-89f3-6707df2a16a4	C8344	R074	card	delivered	70.4
6	22accf1f-f43d-4051-9240-9a971c27bf85	C3024	R039	card	delivered	54.82
7	bee913bc-f4fd-44b7-8f51-229c93ef4c5d	C2294	R018	card	delivered	18.34
8	5d838bcb-e367-4734-93fe-c9a2551e440f	C8811	R190	card	delivered	68.42
9	938839a3-20ac-4b3b-a9b1-fb9b364bc210	C1179	R159	card	delivered	51.5
10	3cd888ae-1614-4d75-bd5c-037edd3684f4	C2504	R110	card	delivered	118.6

4. Which cuisines generate the highest average order value?
   Hint: Join orders, restaurants, group by cuisine, and calculate average total_amount.

   Code:

```
▶  ✓ 05:50 PM (2s)                                                    4

-- 4. Which cuisines generate the highest average order value?
-- Hint: Join orders, restaurants, group by cuisine, and calculate average total_amount.
SELECT
    r.cuisines,
    ROUND(AVG(o.total_amount), 2) AS average_order_value
FROM sathvika_dinedash_db.sathvika_orders_bronze o
JOIN sathvika_dinedash_db.sathvika_restaurants_bronze r
    ON o.restaurant_id = r.restaurant_id
GROUP BY r.cuisines
ORDER BY average_order_value DESC;

▸ (3) Spark Jobs
```

   Output:

> _sqldf: pyspark.sql.dataframe.DataFrame = [cuisines: string, average_order_value: double]

Table +

	cuisines	average_order_value
1	Thai,Chinese	160.1
2	Thai,Italian,Chinese	156.68
3	Mexican,American	146.19
4	Thai,Mexican	144.46
5	Mexican,American,Thai	143.95
6	Thai,Indian,American	129.59
7	American,Mexican	123.96
8	Thai,American,Mexican	121.78
9	Italian,Indian,Mexican	121.05
10	American,Chinese	118.95

5. Which delivery agents have delivered the most high-value orders (above $50)?
   Hint: Filter orders for total_amount > 50, join with agent, group by agent_id, count orders.

   Code:

```
05:52 PM (1s)                                                    5

-- 5. Which delivery agents have delivered the most high-value orders (above $50)?
-- Hint: Filter orders for total_amount > 50, join with agent, group by agent_id, count orders.

SELECT
    o.agent_id,
    a.name,
    COUNT(*) AS order_count
FROM sathvika_dinedash_db.sathvika_orders_bronze o
JOIN sathvika_dinedash_db.sathvika_delivery_agents_bronze a
    ON o.agent_id = a.agent_id
WHERE o.total_amount > 50
GROUP BY o.agent_id, a.name
ORDER BY order_count DESC;

(3) Spark Jobs
```

Output:

_sqldf: pyspark.sql.dataframe.DataFrame = [agent_id: string, name: string ... 1 more field]

Table +

	agent_id	name	order_count
1	A309	Christopher Chase	213
2	A207	Christopher Williams	171
3	A393	Justin Lam	118
4	A350	Charlene Morgan	112
5	A370	Joan Harrison	110
6	A138	Carrie Maxwell	108
7	A178	John King	108
8	A360	Justin Smith	106
9	A168	James Osborn	104
10	A245	Teresa Ryan	102

6. Which cities (area_name) are generating the most revenue for Italian restaurants?

   Hint: Join orders, restaurants, locations, filter cuisine = 'Italian', group by area_name, sum total_amount.

   Code:

```
✓ 05:56 PM (2s)                                                    6

-- 6. Which cities (area_name) are generating the most revenue for Italian restaurants?
-- Hint: Join orders, restaurants, locations, filter cuisine = 'Italian', group by area_name, sum total_amount.

SELECT
    l.area AS area_name,
    ROUND(SUM(o.total_amount), 2) AS revenue
FROM sathvika_dinedash_db.sathvika_orders_bronze o
JOIN sathvika_dinedash_db.sathvika_restaurants_bronze r
    ON o.restaurant_id = r.restaurant_id
JOIN sathvika_dinedash_db.sathvika_locations_bronze l
    ON r.location_id = l.location_id
WHERE r.cuisines LIKE '%Italian%'
GROUP BY l.area
ORDER BY revenue DESC;

▸ (4) Spark Jobs
```

Output:

> _sqldf: pyspark.sql.dataframe.DataFrame = [area_name: string, revenue: double]

Table +

	area_name	revenue
1	Padilla Mall	27277.6
2	Daryl Extensions	17360.82
3	Brennan Center	16184.11
4	Harris Grove	16109.65
5	Boyle Dam	15221.04
6	Montgomery Gateway	14807.66
7	Smith Rapids	11402.97
8	Torres Mountain	11368.6
9	Henderson Creek	11356.79
10	Rich Expressway	10652.45
11	Adams Fort	10640.59
12	Kevin Cove	10048.1
13	Katherine Island	9753.81
14	Smith Garden	9126.28
15	Simon Circles	9118.09

35 rows | 1.757s runtime

7. What is the tip-to-total ratio by cuisine type?
   Hint: Join orders, restaurants, group by cuisine, calculate average tip / total_amount.

   Code:

```
▶  ✓ 06:11 PM (1s)                                                    7

-- 7. What is the tip-to-total ratio by cuisine type?
-- Hint: Join orders, restaurants, group by cuisine, calculate average tip / total_amount.

SELECT
    r.cuisines,
    SUM(o.tip) / SUM(o.total_amount) AS tip_to_total_ratio
FROM sathvika_dinedash_db.sathvika_orders_bronze o
JOIN sathvika_dinedash_db.sathvika_restaurants_bronze r
    ON o.restaurant_id = r.restaurant_id
GROUP BY r.cuisines
ORDER BY tip_to_total_ratio DESC;

▸ (3) Spark Jobs
```

Output:

_sqldf: pyspark.sql.dataframe.DataFrame = [cuisines: string, tip_to_total_ratio: double]

Table +

	cuisines	tip_to_total_ratio
1	Chinese,American	0.1630295032439566
2	Indian,Mexican,American	0.1433364926011985
3	Thai,American,Indian	0.09289116111708078
4	Chinese,Italian,Thai	0.08002964201565706
5	Mexican,Italian,American	0.07546926003361999
6	American,Italian,Indian	0.06287668787668786
7	Italian,Thai,Chinese	0.05765953938327928
8	Italian,Chinese,American	0.05066045436142366
9	Chinese,Indian,American	0.04965877495997978
10	Italian,Mexican	0.0452337176353529
11	Italian	0.04444089742834904
12	Chinese	0.04218793035371627
13	Chinese,Thai	0.04206092758796065
14	Mexican,Italian,Thai	0.041557603686635934
15	American	0.041118886014299846

75 rows | 0.890s runtime

8. Which menu items are most frequently ordered, and from which restaurants?

   Hint: Explode items_ordered in orders, join with menu_items, group by item_name, restaurant_id, count.

   Code:

```
▶  ✓ 06:17 PM (2s)                                                    8

-- 8. Which menu items are most frequently ordered, and from which restaurants?
-- Hint: Explode items_ordered in orders, join with menu_items, group by item_name, restaurant_id, count.

SELECT
    m.item_name,
    sub.restaurant_id,
    COUNT(*) AS order_count
FROM (
    SELECT o.restaurant_id, item.item_id
    FROM sathvika_dinedash_db.sathvika_orders_bronze o
    LATERAL VIEW EXPLODE(o.items_ordered) t AS item
) sub
JOIN sathvika_dinedash_db.sathvika_menu_items_bronze m
    ON sub.item_id = m.item_id
GROUP BY m.item_name, sub.restaurant_id
ORDER BY order_count DESC;

▸ (3) Spark Jobs
```

Output:

_sqldf: pyspark.sql.dataframe.DataFrame = [item_name: string, restaurant_id: string ... 1 more field]

Table +

	item_name	restaurant_id	order_count
1	Argue Cake	R198	111
2	Purpose Brownie	R138	104
3	Even Tea	R090	103
4	Rich Biryani	R139	101
5	Painting Coffee	R084	101
6	Sell Ice Cream	R039	101
7	Others Wrap	R165	100
8	Finish Garlic Bread	R082	100
9	Market Spring Roll	R102	99
10	Student Tea	R158	99
11	Bring Cake	R100	98
12	Thought Pizza	R005	98
13	Street Pizza	R011	98
14	Newspaper Tea	R141	97
15	Very Soda	R131	97

9. Which agents have delivered the most diverse cuisines?
   Hint: Join orders, restaurants, group by agent_id, count distinct cuisine.

Code:

```
▶  ✓ 06:19 PM (2s)                                                    9

-- 9. Which agents have delivered the most diverse cuisines?
-- Hint: Join orders, restaurants, group by agent_id, count distinct cuisine.

SELECT
    o.agent_id,
    COUNT(DISTINCT r.cuisines) AS cuisine_count
FROM sathvika_dinedash_db.sathvika_orders_bronze o
JOIN sathvika_dinedash_db.sathvika_restaurants_bronze r
    ON o.restaurant_id = r.restaurant_id
GROUP BY o.agent_id
ORDER BY cuisine_count DESC;

▸ (4) Spark Jobs
```

Output:

> _sqldf: pyspark.sql.dataframe.DataFrame = [agent_id: string, cuisine_count: long]

Table +

	agent_id	cuisine_count
1	A309	49
2	A168	45
3	A360	44
4	A245	44
5	A207	44
6	A393	43
7	A351	42
8	A331	42
9	A355	41
10	A258	41
11	A320	40
12	A212	40
13	A138	40
14	A056	39
15	A178	39

400 rows | 1.651s runtime

10. What is the average customer lifetime value by signup year?
    Hint: Join orders, customers, extract year from signup_date, group by year, sum total_amount per customer, then average total amount per customer.

Code:

```
06:21 PM (2s)                                                    10
-- 10. What is the average customer lifetime value by signup year?
-- Hint: Join orders, customers, extract year from signup_date, group by year, sum total_amou
then average.

SELECT
    signup_year,
    ROUND(AVG(customer_total), 2) AS average_customer_lifetime_value
FROM (
    SELECT
        c.customer_id,
        YEAR(c.signup_date) AS signup_year,
        SUM(o.total_amount) AS customer_total
    FROM sathvika_dinedash_db.sathvika_orders_bronze o
    JOIN sathvika_dinedash_db.sathvika_customers_bronze c
        ON o.customer_id = c.customer_id
    GROUP BY c.customer_id, YEAR(c.signup_date)
)
GROUP BY signup_year
ORDER BY signup_year;
▸ (4) Spark Jobs
```

Output:

_sqldf: pyspark.sql.dataframe.DataFrame = [signup_year: integer, average_customer_lifetime_value: double]

Table +

	signup_year	average_customer_lifetime_value
1	1954	22.64
2	1955	172.65
3	1956	139.9
4	1957	155.41
5	1958	135.53
6	1959	141.08
7	1960	173.02
8	1961	222.25
9	1962	192.06
10	1963	137.26

11. Which restaurants have the highest order frequency per location?

Hint: Join orders, restaurants, locations, group by restaurant_id, location_id, count orders.

Code:

```
06:22 PM (2s)                                                    11

-- 11. Which restaurants have the highest order frequency per location?
-- Hint: Join orders, restaurants, locations, group by restaurant_id, location_id, count orders.

SELECT
    o.restaurant_id,
    r.location_id,
    l.area,
    COUNT(*) AS order_count
FROM sathvika_dinedash_db.sathvika_orders_bronze o
JOIN sathvika_dinedash_db.sathvika_restaurants_bronze r
    ON o.restaurant_id = r.restaurant_id
JOIN sathvika_dinedash_db.sathvika_locations_bronze l
    ON r.location_id = l.location_id
GROUP BY
    o.restaurant_id,
    r.location_id,
    l.area
ORDER BY order_count DESC;

(4) Spark Jobs
```

Output:

_sqldf: pyspark.sql.dataframe.DataFrame = [restaurant_id: string, location_id: string ... 2 more fields]

Table +

	restaurant_id	location_id	area	order_c...
1	R005	L039	Elizabeth Locks	114
2	R198	L022	White Tunnel	111
3	R031	L015	Hunt Drives	110
4	R083	L069	Reed Glens	108
5	R171	L093	Copeland Parks	108
6	R131	L046	Reynolds Curve	108
7	R058	L031	Edwards Plains	107
8	R092	L057	Kimberly Points	106
9	R138	L018	Hill Station	104
10	R151	L065	Nicholas Fork	104

12. Which locations are prone to order cancellations?

Hint: Filter orders with order_status = 'cancelled', join locations using delivery_location_id, group by area_name, count.

Code:

```
06:27 PM (1s)
-- 12. Which locations are prone to order cancellations?
-- Hint: Filter orders with order_status = 'cancelled', join locations

SELECT
    l.area AS area_name,
    COUNT(*) AS cancelled_orders
FROM sathvika_dinedash_db.sathvika_orders_bronze o
JOIN sathvika_dinedash_db.sathvika_locations_bronze l
    ON o.delivery_location_id = l.location_id
WHERE o.order_status = 'cancelled'
GROUP BY l.area
ORDER BY cancelled_orders DESC;

(3) Spark Jobs
```

Output:

_sqldf: pyspark.sql.dataframe.DataFrame = [area_name: string, cancelled_orders: long]

Table

	area_name	cancelled_orders
1	Evans Stream	58
2	Elizabeth Locks	57
3	Flores Vista	55
4	Ashley Extensions	53
5	Padilla Mall	52
6	Hill Station	52
7	Curtis Oval	51
8	White Tunnel	51
9	Rodriguez Mill	51
10	Joshua Drive	51
11	Arnold Fort	50
12	Carpenter Views	49
13	Hunt Drives	49
14	John Underpass	49
15	Lance Radial	49

100 rows | 1.028s runtime

13. Find top 10 customers who have ordered from the widest variety of restaurants.

Hint: Join orders, group by customer_id, count distinct restaurant_id, sort and pick top 10.

Code:

```
06:27 PM (1s)                                                    13

-- 13. Find top 10 customers who have ordered from the widest variety of restaurants.
-- Hint: Join orders, group by customer_id, count distinct restaurant_id, sort and pick top 10.

SELECT
    customer_id,
    COUNT(DISTINCT restaurant_id) AS restaurant_count
FROM sathvika_dinedash_db.sathvika_orders_bronze
GROUP BY customer_id
ORDER BY restaurant_count DESC
LIMIT 10;

(3) Spark Jobs
```

Output:

_sqldf: pyspark.sql.dataframe.DataFrame = [customer_id: string, restaurant_count: long]

Table +

	customer_id	restaurant_count
1	C7381	9
2	C6444	8
3	C2231	8
4	C0362	8
5	C1449	8
6	C6672	7
7	C6971	7
8	C4867	7
9	C1423	7
10	C9530	7

10 rows | 1.307s runtime

14. Identify restaurants with frequent repeat customers.

Hint: Join orders, group by restaurant_id, customer_id, count, filter where count > 1.

Code:

```
06:02 PM (4s)                                                    2

SELECT
    r.restaurant_id,
    r.name AS restaurant_name,
    o.customer_id,
    COUNT(DISTINCT o.order_id) AS order_count
FROM sathvika_dinedash_db.sathvika_orders_bronze o
JOIN sathvika_dinedash_db.sathvika_restaurants_filtered r
    ON o.restaurant_id = r.restaurant_id
GROUP BY
    r.restaurant_id,
    r.name,
    o.customer_id
HAVING COUNT(DISTINCT o.order_id) > 1
ORDER BY order_count DESC;

▸ (4) Spark Jobs
```

Output:

_sqldf: pyspark.sql.dataframe.DataFrame = [restaurant_id: string, restaurant_name: string ... 2 more fields]

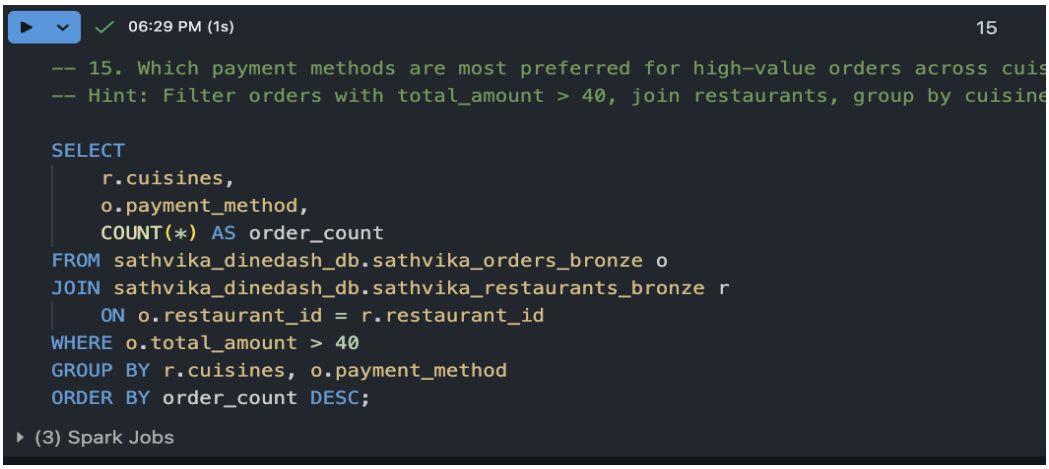
Table +

	restaurant_id	restaurant_name	customer_id	order_count
1	R141	Food Haven Grill	C7746	2
2	R179	Spice Villa Grill	C5469	2
3	R181	Spice Villa Hub	C3734	2
4	R053	Tasty Bites Express	C2179	2
5	R041	Curry Leaf Express	C6269	2
6	R012	Pasta Palace Hub	C0232	2
7	R100	Curry Leaf Corner	C0011	2
8	R038	Spice Villa Hub	C9577	2
9	R102	Pasta Palace Express	C6672	2
10	R196	Spice Villa Kitchen	C0113	2

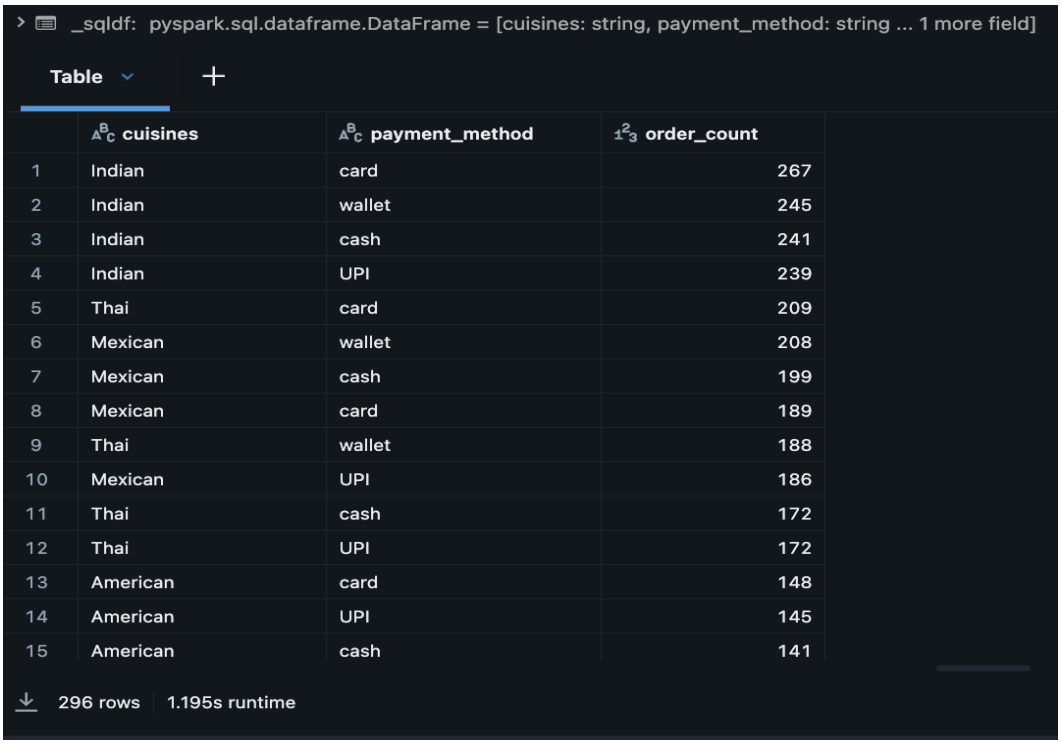
15. Which payment methods are most preferred for high-value orders across cuisines?

    Hint: Filter orders with total_amount > 40, join restaurants, group by cuisine, payment_method, count.

Code:



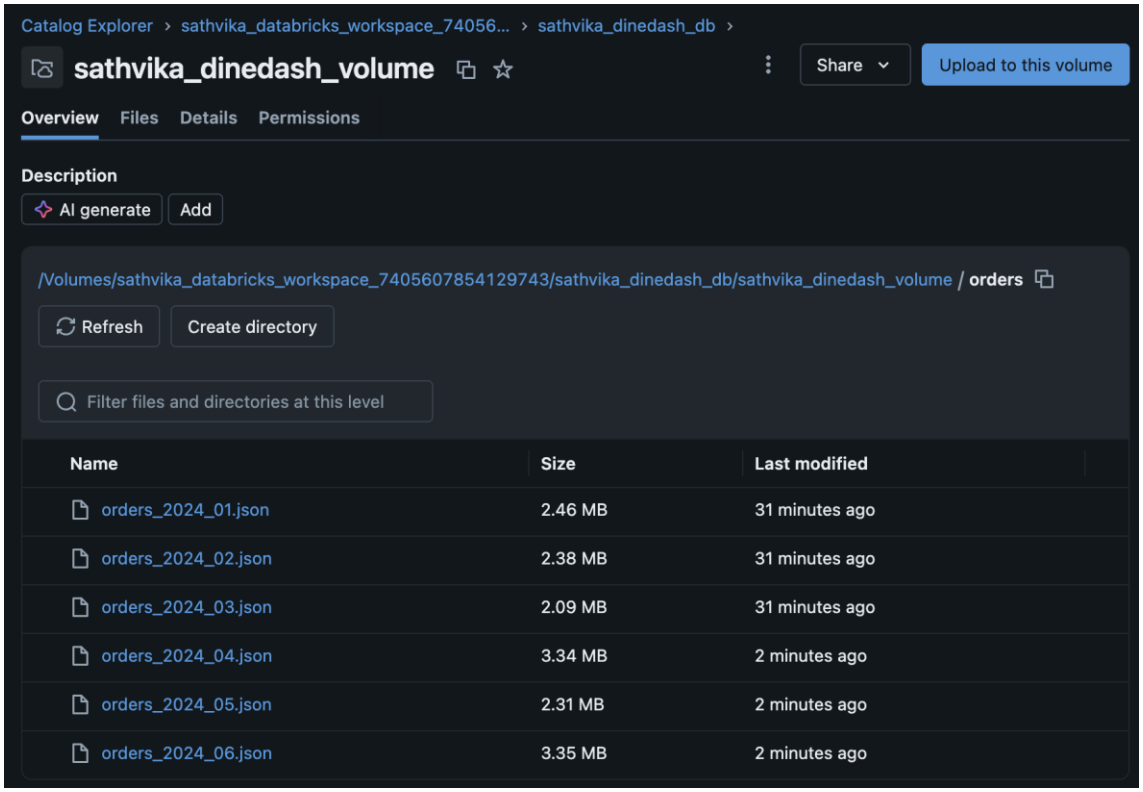
Output:



	cuisines	payment_method	order_count
1	Indian	card	267
2	Indian	wallet	245
3	Indian	cash	241
4	Indian	UPI	239
5	Thai	card	209
6	Mexican	wallet	208
7	Mexican	cash	199
8	Mexican	card	189
9	Thai	wallet	188
10	Mexican	UPI	186
11	Thai	cash	172
12	Thai	UPI	172
13	American	card	148
14	American	UPI	145
15	American	cash	141

296 rows | 1.195s runtime

Add, the Reaming April to June json files to the volume



Creating delta live tables:

BRONZE LAYER: Ingest Raw Data

```
CREATE OR REFRESH STREAMING TABLE sath_orders_bronze
AS
SELECT *
FROM STREAM read_files(
'/Volumes/sathvika_databricks_workspace_7405612503983294/sathvika_dinedash_db/sathvika_dinedash_volume/orders/',
  format => 'json'
);


CREATE OR REFRESH MATERIALIZED VIEW sath_restaurants_bronze AS
SELECT *
FROM read_files(
'/Volumes/sathvika_databricks_workspace_7405612503983294/sathvika_dinedash_db/sathvika_dinedash_volume/restaurants/dim_restaurants.csv',
format=>'csv',
inferSchema => true,
header => true
);


CREATE OR REFRESH MATERIALIZED VIEW sath_items_bronze AS
SELECT *
```

```
FROM read_files(
'/Volumes/sathvika_databricks_workspace_7405612503983294/sathvika_dinedash_db/sathvika_dinedash_volume/menu_items/dim_menu_items.csv',
format=>'csv',
inferSchema => true,
header => true
);


CREATE OR REFRESH MATERIALIZED VIEW sath_customers_bronze AS
SELECT *
FROM read_files(
'/Volumes/sathvika_databricks_workspace_7405612503983294/sathvika_dinedash_db/sathvika_dinedash_volume/customers/dim_customers.csv',
format=>'csv',
inferSchema => true,
header => true
);


CREATE OR REFRESH MATERIALIZED VIEW sath_agents_bronze AS
SELECT *
FROM read_files(
'/Volumes/sathvika_databricks_workspace_7405612503983294/sathvika_dinedash_db/sathvika_dinedash_volume/delivery_agents/dim_delivery_agents.csv',
format=>'csv',
inferSchema => true,
header => true
);


CREATE OR REFRESH MATERIALIZED VIEW sath_locations_bronze AS
SELECT *
FROM read_files(
'/Volumes/sathvika_databricks_workspace_7405612503983294/sathvika_dinedash_db/sathvika_dinedash_volume/locations/dim_locations.csv',
format=>'csv',
inferSchema => true,
header => true
);
```

SILVER LAYER: Clean + Normalize

```
CREATE OR REFRESH STREAMING LIVE TABLE sath_cleaned_orders (
CONSTRAINT valid_order_id
EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,

CONSTRAINT valid_customer
EXPECT (customer_id IS NOT NULL) ON VIOLATION DROP ROW,
```

```
CONSTRAINT valid_items
EXPECT (size(items_ordered) > 0) ON VIOLATION DROP ROW
)
AS
SELECT
        order_id,
        timestamp AS order_ts,
        customer_id,
        restaurant_id,
        agent_id,
        delivery_location_id,
        items_ordered,
        explode(items_ordered) AS item,
        total_amount,
        tip,
        payment_method,
        order_status
FROM STREAM(Live.sath_orders_bronze);


CREATE OR REFRESH STREAMING TABLE sath_orders_silver
(
CONSTRAINT valid_restaurant_name
EXPECT (restaurant_name IS NOT NULL) ON VIOLATION DROP ROW,

CONSTRAINT valid_agent
EXPECT (agent_name IS NOT NULL) ON VIOLATION DROP ROW,

CONSTRAINT valid_item
EXPECT (item_id IS NOT NULL) ON VIOLATION DROP ROW
)
AS
SELECT
        o.order_id,
        o.order_ts,
        o.customer_id,
        c.name AS customer_name,
        c.email,
        o.restaurant_id,
        r.name AS restaurant_name,
        r.cuisines,
        r.rating AS restaurant_rating,
        o.agent_id,
        a.name AS agent_name,
        a.rating AS agent_rating,
        o.delivery_location_id,
        l.city,
        o.item.item_id AS item_id,
        i.item_name,
        i.category,
```

```
        o.item.price AS item_price,
        o.item.quantity AS item_quantity,
        o.total_amount,
        o.tip,
        o.payment_method,
        o.order_status
FROM STREAM(Live.sath_cleaned_orders) o
LEFT JOIN Live.sath_customers_bronze c
        ON o.customer_id = c.customer_id
LEFT JOIN Live.sath_restaurants_bronze r
        ON o.restaurant_id = r.restaurant_id
LEFT JOIN Live.sath_agents_bronze a
        ON o.agent_id = a.agent_id
LEFT JOIN Live.sath_items_bronze i
        ON o.item.item_id = i.item_id
LEFT JOIN Live.sath_locations_bronze l
        ON o.delivery_location_id = l.location_id;


CREATE OR REFRESH STREAMING TABLE sath_orders_with_agents AS
SELECT
        o.order_id,
        o.order_ts,
        o.agent_id,
        a.name AS agent_name,
        a.rating AS agent_rating,
        o.tip
FROM STREAM(Live.sath_cleaned_orders) o
LEFT JOIN sath_agents_bronze a
        ON o.agent_id = a.agent_id;


CREATE OR REFRESH STREAMING TABLE sath_orders_with_items AS
SELECT
        o.order_id,
        o.order_ts,
        o.restaurant_id,
        exploded.item_id,
        i.item_name,
        i.category,
        exploded.price,
        exploded.quantity
FROM (
        SELECT order_id, timestamp AS order_ts, restaurant_id, explode(items_ordered)
        AS exploded
        FROM STREAM(Live.sath_orders_bronze)
) o
LEFT JOIN sath_items_bronze i
        ON o.exploded.item_id = i.item_id;
```

```
CREATE OR REFRESH STREAMING TABLE sath_orders_with_locations AS
SELECT
        o.order_id,
        o.order_ts,
        o.delivery_location_id,
        l.city,
        l.state,
        l.area
FROM STREAM(Live.sath_cleaned_orders) o
LEFT JOIN sath_locations_bronze l
        ON o.delivery_location_id = l.location_id;
```

GOLD LAYER: Insightful Metrics

1. Total orders per restaurant

```
CREATE OR REFRESH MATERIALIZED VIEW sath_total_orders_per_restaurant AS
SELECT restaurant_name, COUNT(DISTINCT order_id) AS total_orders
FROM sath_orders_silver
GROUP BY restaurant_name;
```

2. Top-selling items

```
CREATE OR REFRESH MATERIALIZED VIEW sath_top_selling_items AS
SELECT item_name, SUM(item_quantity) AS total_qty_sold
FROM sath_orders_silver
GROUP BY item_name
ORDER BY total_qty_sold DESC
LIMIT 10;
```

3. Avg delivery fee and tip by city

```
CREATE OR REFRESH MATERIALIZED VIEW sath_avg_fees_by_city AS
SELECT city, AVG(restaurant_rating) AS avg_restaurant_rating, AVG(tip) AS avg_tip
FROM sath_orders_silver
GROUP BY city;
```

4. Agent performance

```
CREATE OR REFRESH MATERIALIZED VIEW sath_agent_order_counts AS
SELECT agent_name, COUNT(DISTINCT order_id) AS orders_handled,
AVG(agent_rating) AS avg_rating
FROM sath_orders_silver
GROUP BY agent_name
ORDER BY orders_handled DESC;
```

5. Revenue trend

```
CREATE OR REFRESH MATERIALIZED VIEW daily_revenue_trend AS
SELECT DATE(order_ts) AS order_date, SUM(total_amount) AS total_revenue
FROM Live.sath_orders_silver
GROUP BY order_date
ORDER BY order_date;
```

6. Customer lifetime value

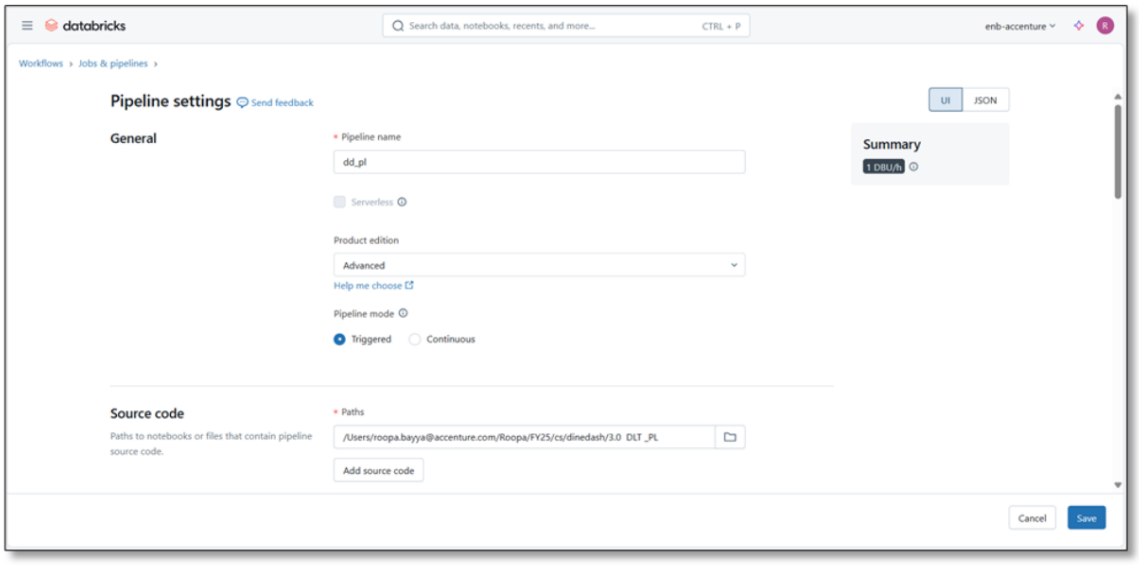
```
CREATE OR REFRESH MATERIALIZED VIEW sath_customer_ltv AS
SELECT customer_id, customer_name, SUM(total_amount) AS lifetime_spend
FROM sath_orders_silver
GROUP BY customer_id, customer_name
ORDER BY lifetime_spend DESC;
```

```
CREATE OR REFRESH STREAMING TABLE sath_orders_with_customers AS
SELECT
        o.order_id,
        o.customer_id,
        c.name AS customer_name,
        c.location_id AS customer_location_id,
        o.total_amount,
        o.tip,
        o.payment_method,
        o.order_status
FROM STREAM(Live.sath_cleaned_orders) o
LEFT JOIN Live.sath_customers_bronze c
        ON o.customer_id = c.customer_id;
```

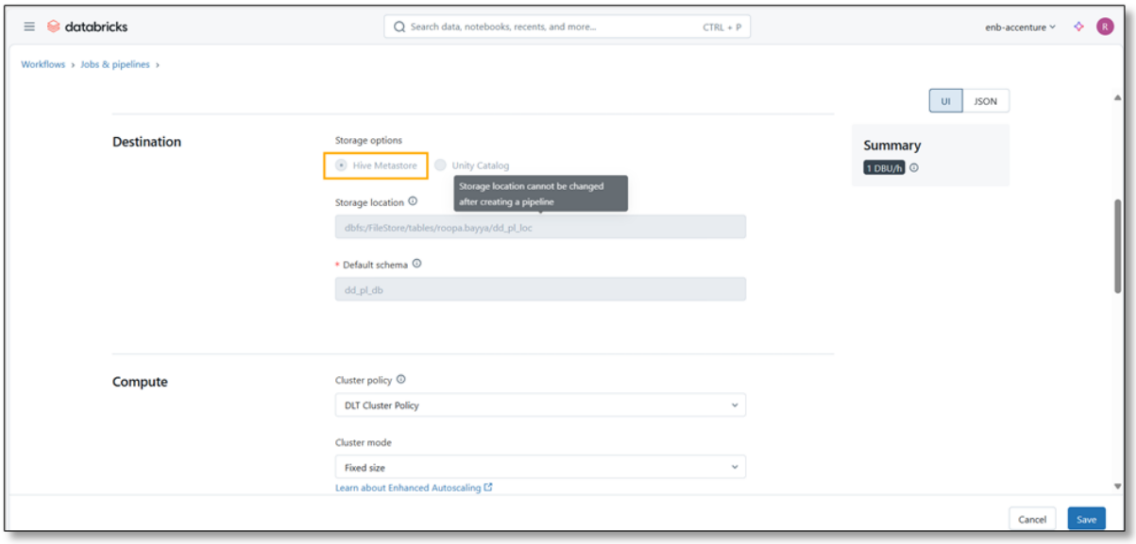
```
CREATE OR REFRESH MATERIALIZED VIEW sath_customer_order_counts AS
SELECT customer_name, COUNT(*) AS total_orders
FROM sath_orders_with_customers
GROUP BY customer_name;
```

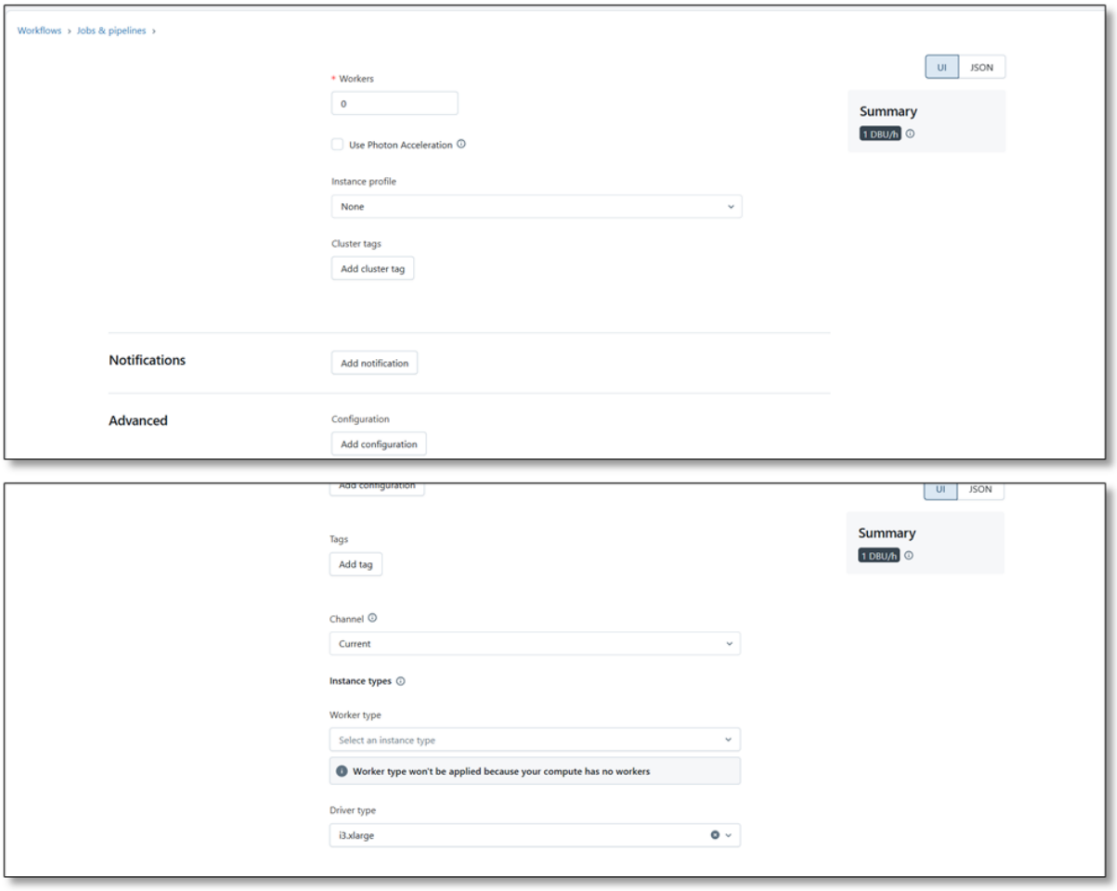
Step 2: Create Pipeline

Goto Workflows menu in Databricks Workspace, select Jobs and Pipelines tab and fill the details as shown below



Ensure that Storage Location and Default schema are unique for you

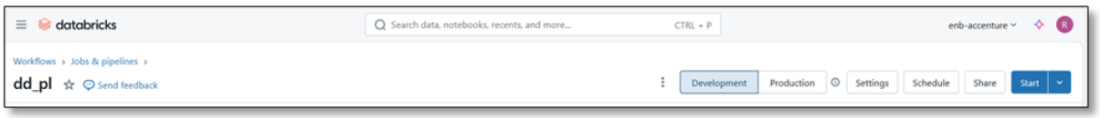




Click on **Create**

Step 3: Start the pipeline manually

Click on **Start**

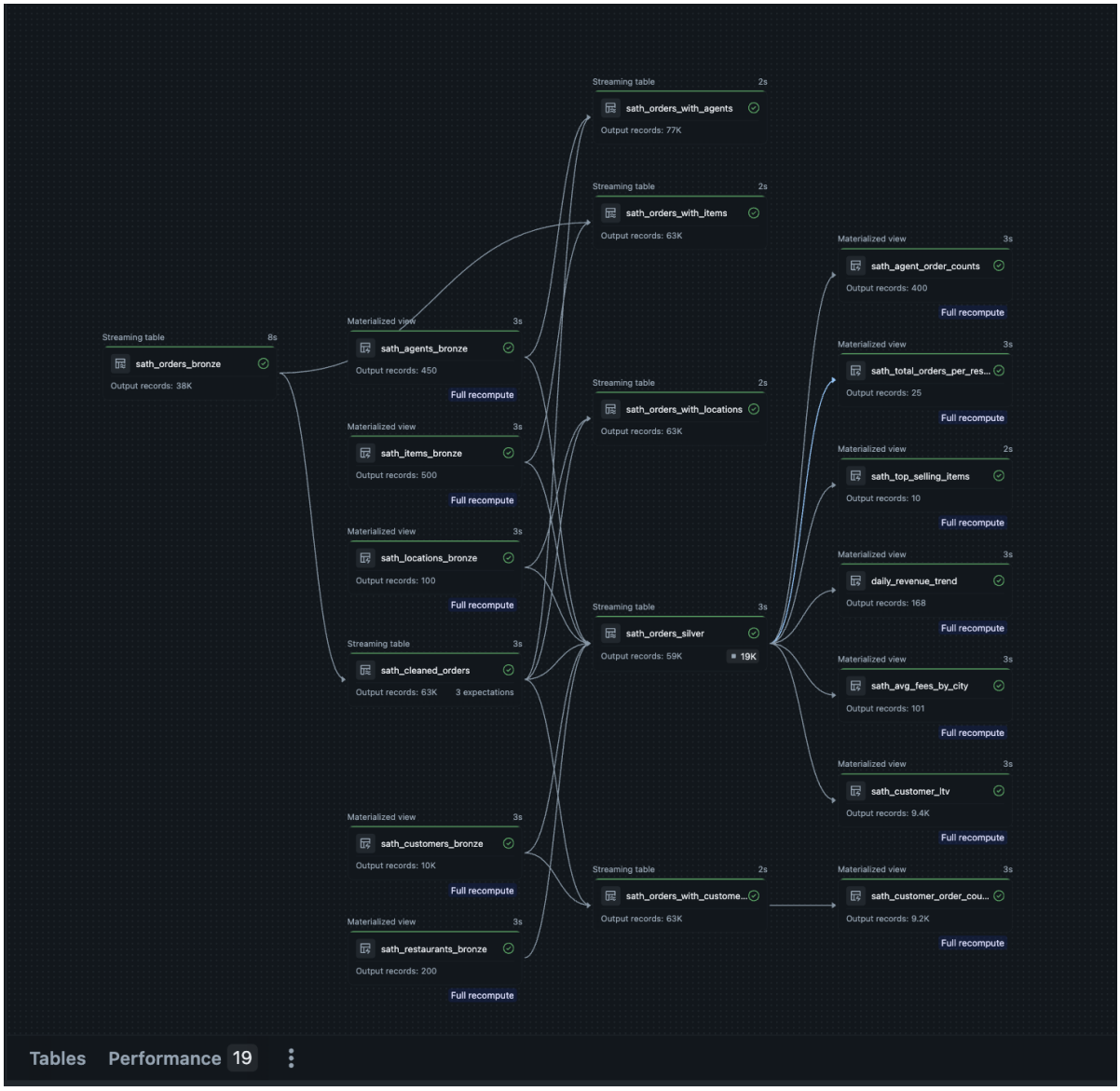


Initially it will take 5-10 for the pipeline to run, as the cluster is being set up.

Once the pipeline is executed, check through the details for each of the table. Below you see the Zoomed out screenshots for the pipeline

PIPELINE EXECUTION (January - June)

The DineDash DLT pipeline was executed incrementally from January to June to process the order data as it became available. Each month's data was added to the source, after which the pipeline processed the new records through the Bronze, Silver, Gold layers. This helped verify that the pipeline could handle incremental data loading and update the downstream tables and business metrics accordingly.



The above diagram shows the complete **Delta Live Tables (DLT) pipeline** for the DineDash project. The pipeline follows a **Bronze → Silver → Gold** architecture to process the order data and generate useful business insights.

1. Bronze Layer – Raw Data

The Bronze layer contains the raw data ingested from the source files. The main orders data is stored in sath_orders_bronze, while the dimension data such as

customers, restaurants, agents, items, and locations is stored separately. This layer preserves the source data before applying transformations.

2. Silver Layer – Cleaned and Enriched Data

The Silver layer cleans and validates the Bronze data. Invalid records are removed using data quality expectations. The order data is then enriched by joining it with customer, restaurant, agent, item, and location information.

The pipeline also creates separate tables such as:

- sath_orders_silver – cleaned and enriched order data
- sath_orders_with_agents – order and delivery-agent information
- sath_orders_with_items – individual ordered items
- sath_orders_with_locations – order and location information
- sath_orders_with_customers – order and customer information

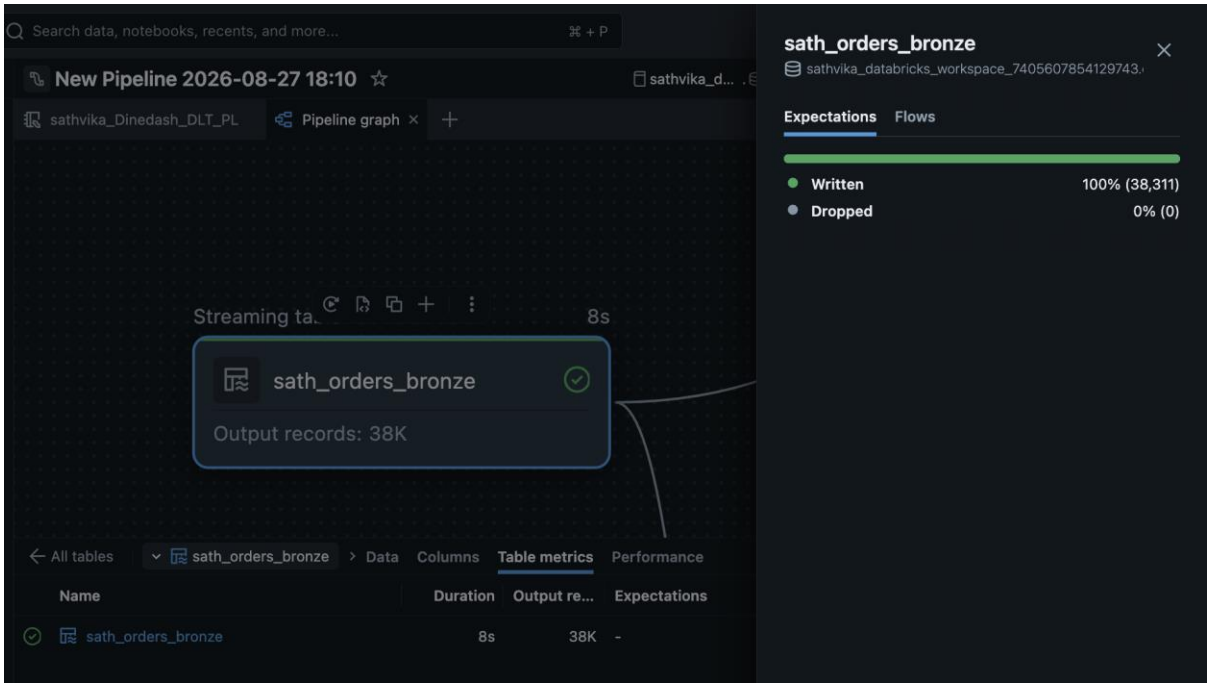
3. Gold Layer – Business Insights

The Gold layer contains the final tables used for analysis and reporting. These tables calculate metrics such as:

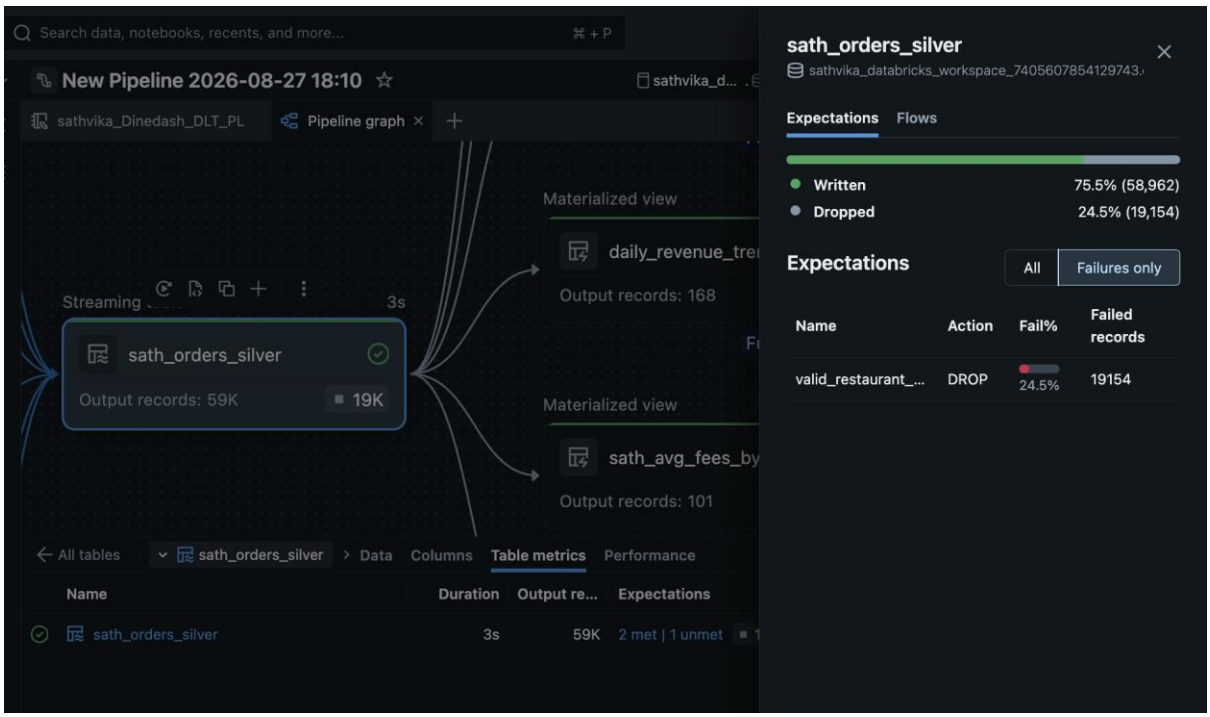
- Total orders per restaurant
- Top-selling items
- Average tip and restaurant rating by city
- Delivery-agent performance
- Daily revenue trends
- Customer lifetime value
- Customer order counts

4. Pipeline Flow

The data flows from the **Bronze layer → Silver layer → Gold layer**. When new order data is added to the source, the pipeline processes the new data through the dependent tables and updates the downstream results.

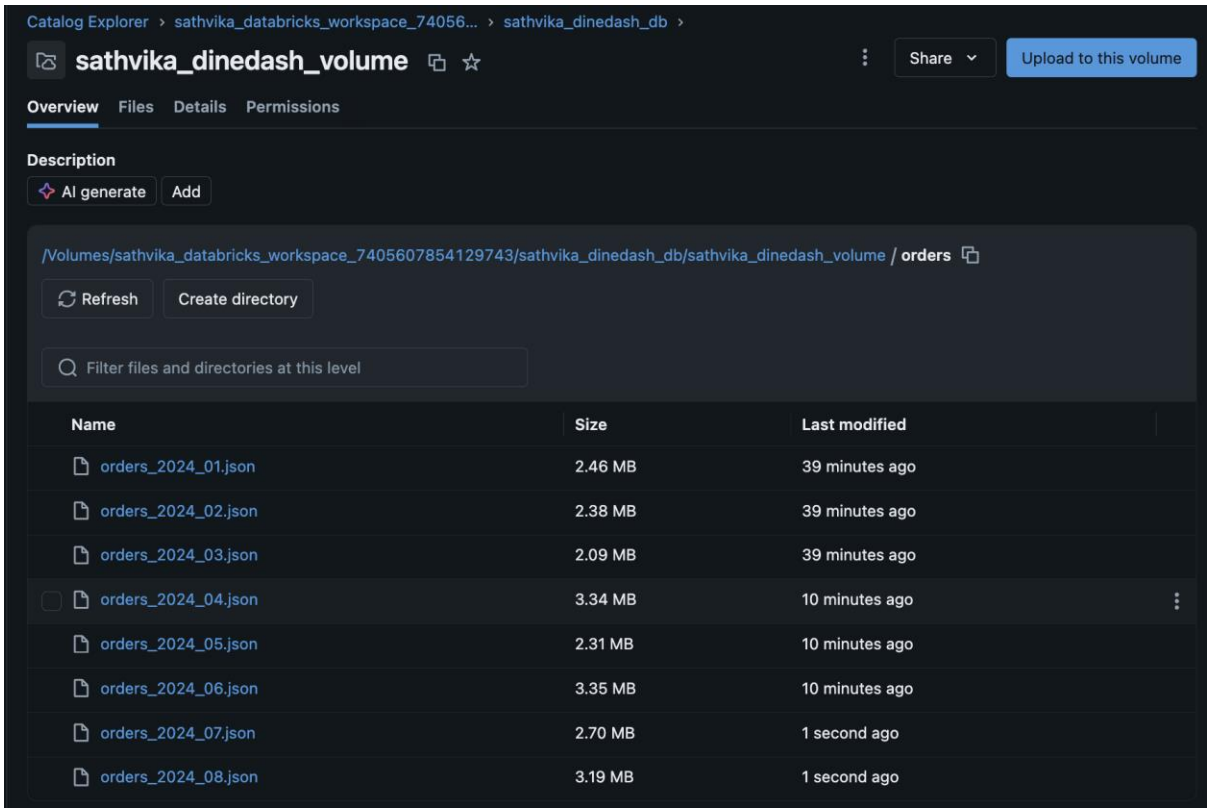


The Bronze streaming table successfully ingested the raw order data, writing **38,311 records** with **0 records dropped**. This confirms that all incoming records passed the Bronze-layer ingestion successfully.



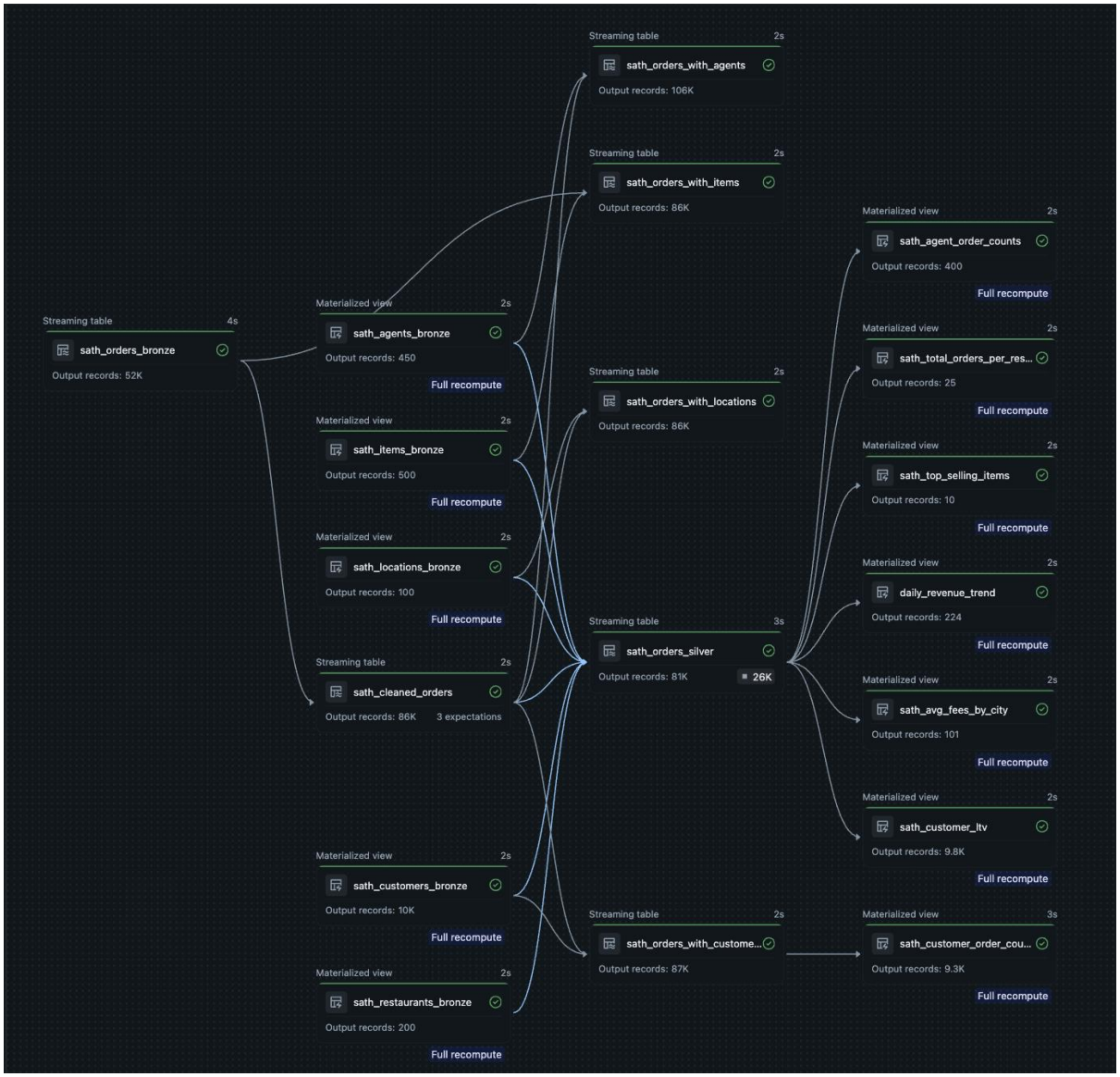
The Silver streaming table processed the Bronze data and applied the defined data-quality expectations. **58,962 records (75.5%) were written**, while **19,154 records (24.5%) were dropped** because they failed the valid_restaurant_name expectation. This demonstrates that invalid records were filtered out during the cleaning stage.

July August Data Ingestion

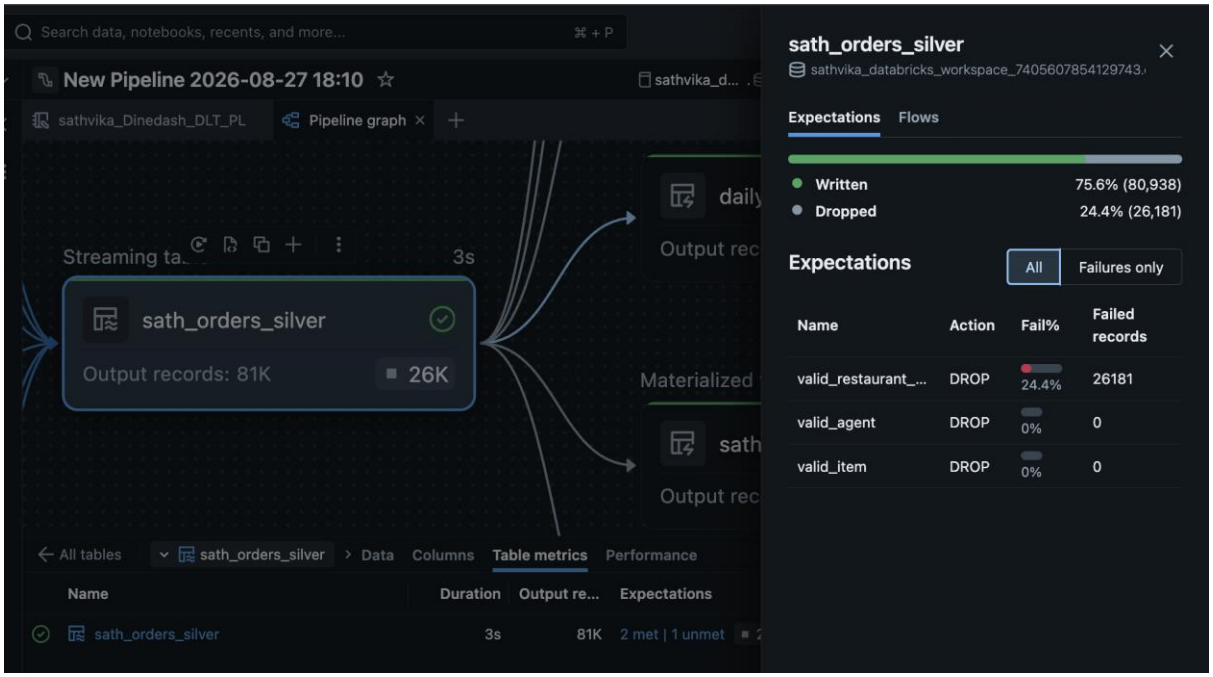


PIPELINE EXECUTION(January - August)

The pipeline was executed again after adding the **July and August data** to the source volume. The new records were incrementally processed through the Bronze, Silver, and downstream tables without recreating the pipeline structure.



After processing the additional July–August data, the pipeline shows an increase in records across the main tables. In particular, the **sath_orders_bronze table increased to approximately 52K records**, while **sath_orders_silver reached approximately 81K output records**. This demonstrates that the pipeline is processing the newly added data along with the existing data and propagating it through the different layers.



QUERIES(JAN-AUGUST)

total_orders_per_restaurant:

```sql
SELECT *
FROM total_orders_per_restaurant;
```

This query displays the restaurant-wise order counts from the total_orders_per_restaurant table to compare the number of orders handled by each restaurant.

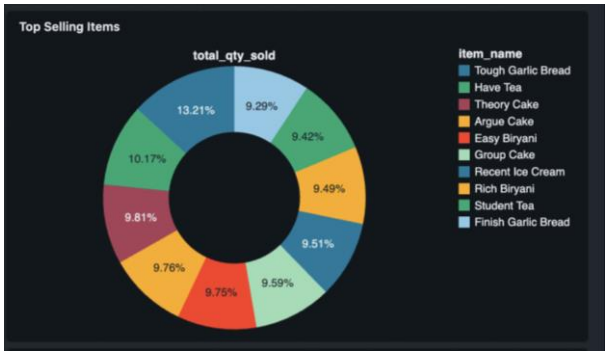


Bar chart — used to compare order counts across different restaurants.

top_selling_items:

```sql
SELECT *
FROM top_selling_items;
```

This query retrieves the top-selling menu items based on the total quantity sold and displays their contribution to overall item sales.

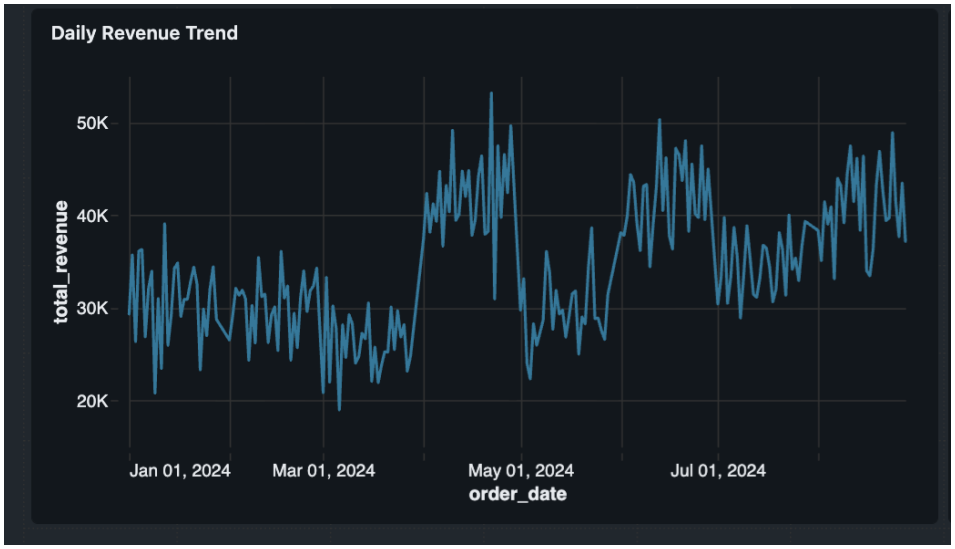


Pie chart - shows the contribution/share of each item.

daily_revenue_trend:

```sql
SELECT *
FROM daily_revenue_trend
ORDER BY order_date;
```

This query retrieves the revenue generated for each day and displays it in chronological order using order_date.



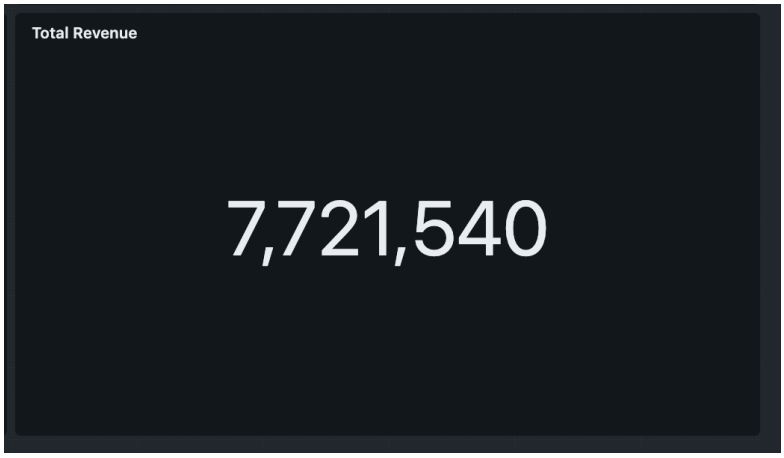
Line chart - showing how revenue changes over time.

total_revenue:

```sql
SELECT
    ROUND(SUM(total_amount), 2) AS total_revenue
FROM orders_silver;

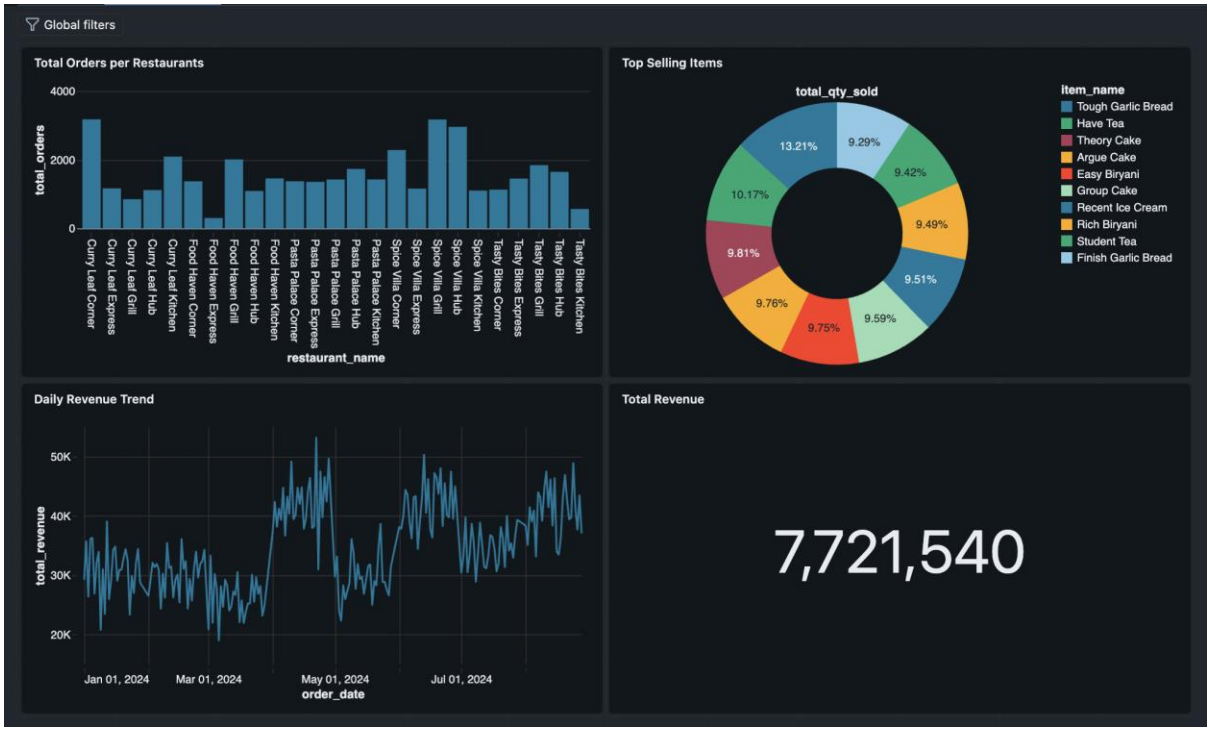
```

This query calculates the total revenue generated from all orders by adding the total_amount values in the orders_silver table.



Counter chart - The total revenue generated from the available order data is $7,721,540.

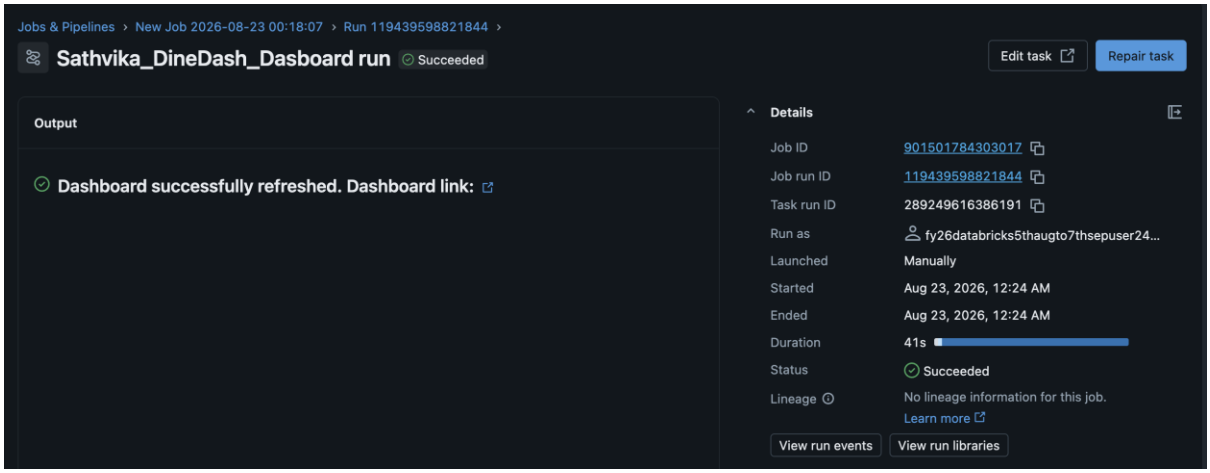
DASHBOARD (JANUARY-AUGUST)

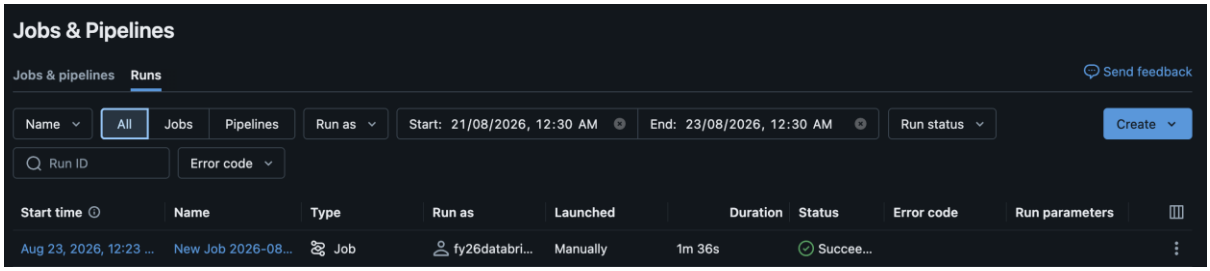


The dashboard brings together the key DineDash metrics, including restaurant-wise orders, top-selling items, daily revenue trends, and total revenue, providing an overall view of business performance from January to August.

JOB CREATION:

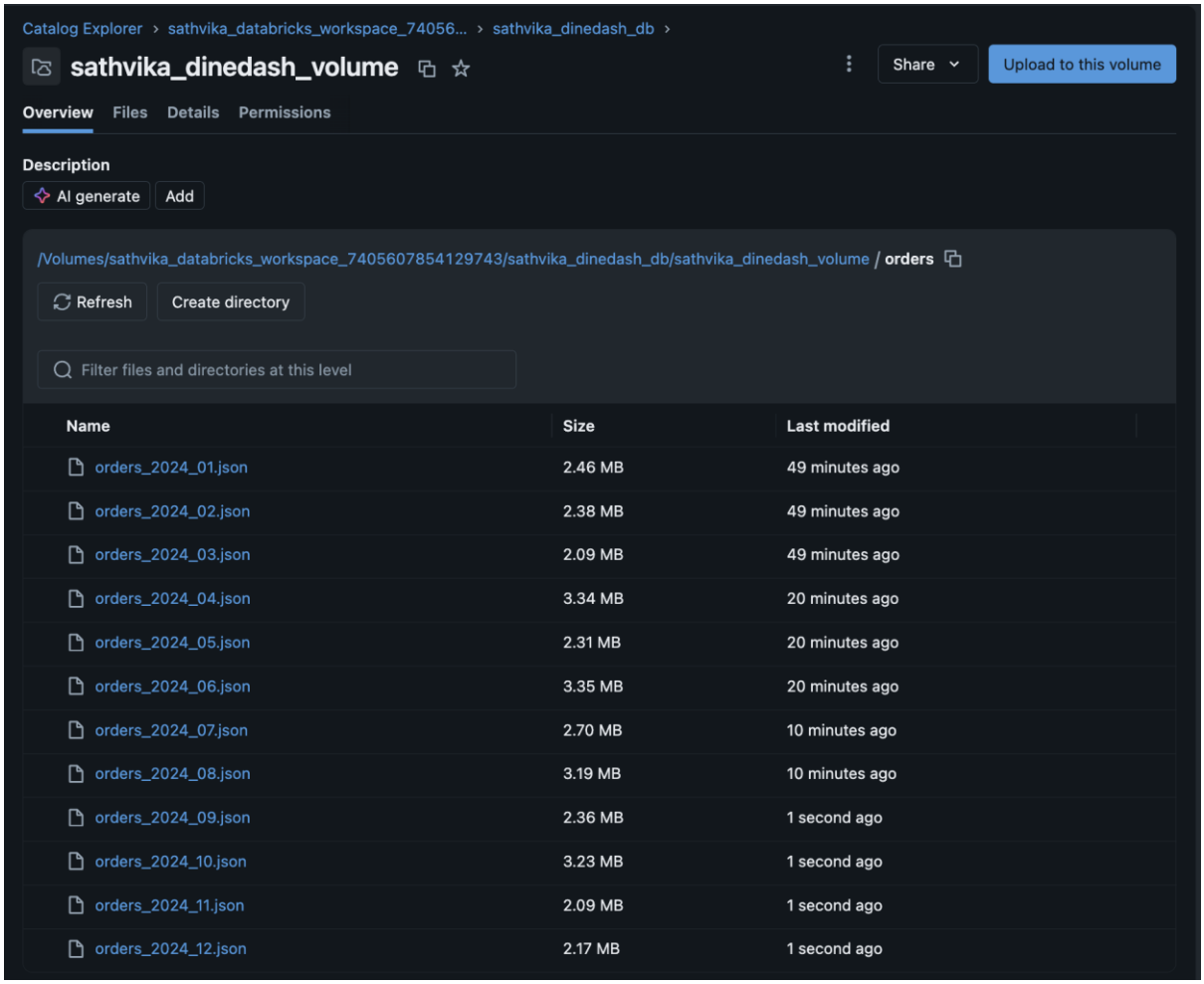






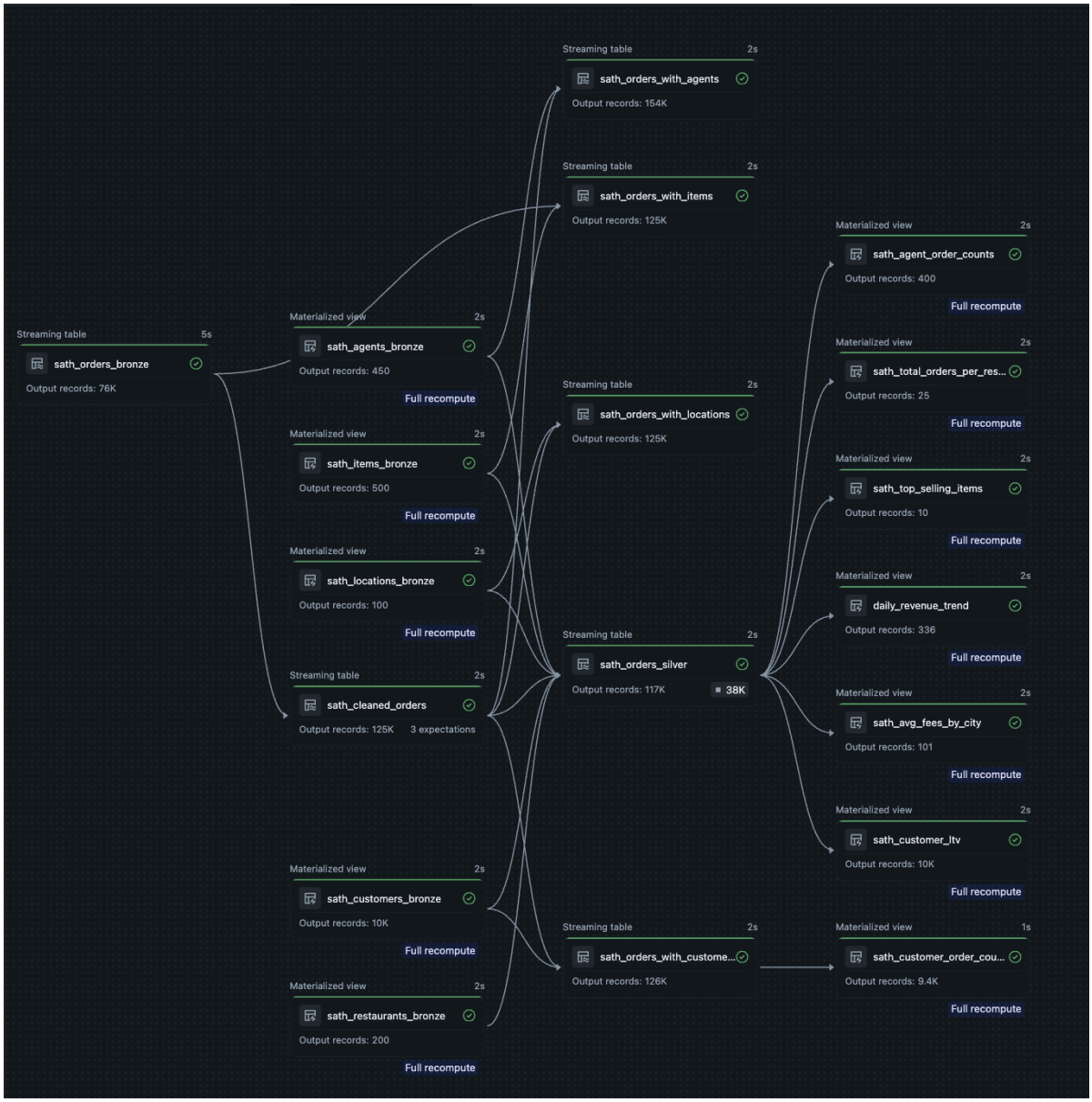
A Databricks job was created to automate the execution of the DineDash pipeline and dashboard, with the successful job run confirming that the pipeline and dashboard workflow executed successfully.

Remaining Data Ingestion:



JOB EXECUTION(January - December)

Pipeline:



The updated pipeline shows the complete flow from the Bronze layer through Silver to the Gold layer after the new data ingestion. The main sath_orders_bronze table has increased to 76K records, while sath_orders_silver contains 117K output records, showing that the pipeline successfully processed the additional data and generated updated downstream metrics.

Dashboard:



The dashboard presents the updated business metrics after ingesting the additional data. The total revenue has increased to 11,151,832, reflecting the additional data processed through the pipeline.